CINAMATE

The Operator's Manual

A PRIVATE STUDIO SYSTEM · 16 CHAPTERS

CONTENTS

Getting Started & The Studio Gate

Cinamate is a private studio system. It has five keys, one door, and twenty-eight rooms behind it. This chapter is about the door: who holds a key, what the lock protects, how long a key stays warm, and what you are looking at once you are through.

> **About this manual.** Everything here was established by reading the source in this repository, not by operating a running site. The platform is under active development and the code moves faster than this page. Where the manual and the software disagree, the software is what runs. Where a behaviour could not be established from source, this chapter says so rather than guessing.

Two doors, and only one of them opens

A deployment publishes two things. The first is a small public shell: the landing page, `login.html`, the brand assets, and a short allow-list of files the landing itself loads. The second is the studio — and the studio is not published at all. It is bundled *inside* a serverless function: at release time `scripts/deploy_cinamate.mjs` moves everything not on that allow-list into the payload of `netlify/functions/gate.js` and rewrites the redirect rules so those paths resolve to it.

That is the security model in one sentence: **there is no "view source" route into Cinamate.** An anonymous visitor asking for a module page is not shown a page with the buttons greyed out — those bytes are not on the public CDN at all. They exist only inside the function, which reads the cookie before it reads the file.

The sign-in page

`login.html` is a single card headed CINAMATE, with the line *Production access — owners only* beneath it, fields labelled Login and Password, and a button reading **Enter the studio**. Submitting posts the name and password as JSON to `/.netlify/functions/verify-owner`. Two query parameters alter it: `?u=` pre-fills the Login field, and `?to=` sets where you land afterwards, defaulting to `/dashboard.html` and refused unless it resolves to a plain same-origin path — so a crafted link cannot use the sign-in card to bounce you off the site.

On success the page keeps only a display record — owner short and timestamp — in `sessionStorage` under `cinamate.session`. It never sees the session token: that is deliberately absent from the response body, because anything this page's script can read, injected script could read too.

The five owner accounts

Five names are valid and no others. The list appears in `verify-owner.js`, `gate.js` and `projects-sync.js`, and all three check it independently.

LOGIN	PASSWORD LIVES IN
`mz465`	`OWNER_PW_MZ465`
`kz465`	`OWNER_PW_KZ465`
`hz465`	`OWNER_PW_HZ465`
`rz465`	`OWNER_PW_RZ465`
`dz465`	`OWNER_PW_DZ465`

The passwords are Netlify environment variables on the deployed site — not in this repository, not in any page, and not in this manual. A sixth, `OWNER_TOKEN_SECRET`, is the HMAC key sessions are signed and verified with.

When sign-in refuses

Every failure returns the same message — *Invalid name or password* — after the same short random pause, whether the name exists or not, so the response cannot be used to discover which logins are real.

Two throttles sit in front of it. The first counts failures per client address: more than **12 in a 10-minute window** answers `429` with *Too many attempts — wait a few minutes and try again*. That count is held in the function's memory and in a shared Netlify Blobs store, `cinamate-auth`, keyed by a digest of the address rather than the address itself. The second counts failures per *name* across every address: past **20** in the same window, answers for that name are delayed by 1.5 seconds. It is a delay and never a lockout — a lockout would let anyone shut an owner out of their own studio by typing a wrong password often enough. Both fail open: if the shared counter cannot be reached the system falls back to the weaker in-memory limit, because a storage outage must not become a denial of service against the only five people who can get in.

The twelve-hour session

A correct password mints a token of the form `owner:name:expires:hmac`, valid for **12 hours**. It returns as two cookies, both with a 43,200-second lifetime:

COOKIE	CARRIES	READABLE BY PAGE SCRIPTS
`cin_owner`	the signed session token	No — HttpOnly
`cin_who`	the owner short, for UI labels only	Yes

On every request the gate re-derives the HMAC, compares it in constant time, checks the expiry, and checks the name against the five. It reads *every* `cin_owner` the request carries, up to twelve, accepting whichever verifies — so a planted cookie cannot stand in front of your real session and quietly sign you out.

When the twelve hours lapse, nothing dramatic happens; things simply stop being served. A page request is redirected to `/login.html?to=` plus the path you were on, so signing in again returns you where you were. A script, stylesheet or image gets a bare `401 Sign in required`. A call to the studio cloud comes back as *Sign in to use the studio cloud*.

> **Two limits of the expiry, both read from source.** There is no renewal path — a token is minted only for a POST carrying the password — so a long day means signing in twice. And a tab left open can look signed in when it is not: the Operator label comes from a `sessionStorage` record whose age nothing checks. Nothing warns you before a session lapses. If a page starts failing to save, reload it; you will be sent to the sign-in card.

The **Exit** control at the foot of the rail clears the display record, expires both cookies, posts a `logout` so the HttpOnly cookie is cleared authoritatively, deletes the service worker's caches so the next person at that browser starts cold, and returns to the front page. That logout is same-origin only and exempt from the throttle: ending a session on a shared machine must never be rate-limited.

The dashboard, and the rail

Signing in lands you on `/dashboard.html`. The status bar carries a per-session identifier of the form `SES-` plus eight hex characters; **Operator**, showing your owner short; a workspace segment naming your local render bridge if `SB_LocalGPU_v1` holds one and otherwise reading *WORKSPACE READY*; and a UTC clock.

Below it is the Producer Suite — portfolio tiles, a productions table, a stage badge per title. These read this browser's own saved module stores, refresh every thirty seconds and whenever the tab regains focus, and stay blank until real work exists. The header stamp reads *LIVE FROM THIS BROWSER*, and it means it: this is the machine you are sitting at, not the studio's whole catalogue.

> **A stale comment to ignore.** `dashboard.html` still carries a header block describing a simulation engine that ticked invented figures upward on timers. The code below it does the opposite and says so. The tiles are real; the comment is a leftover.

Down the left is the rail, and the rail is how you reach everything: twenty-nine entries in six labelled groups, resolving to twenty-eight distinct module pages — *Sales* is a deep link into the budget module rather than a page of its own. Under 900 pixels the rail becomes a bottom tab bar.

GROUP	ENTRIES (PATH)
Develop	Writer `/writer/`, Boards `/boards/`, Clear `/clearance/`, Invest `/investors/`
Prep	Budget `/producer/`, Deals `/contracts/`, Casting `/casting/`, Scout `/locations/`, Wardrobe `/wardrobe/`, Props `/props/`, Sets `/sets/`, Safety `/safety/`
Shoot	Studio `/timeline/`, Office `/production/`, Money `/finance/`, Tools `/tools/`, Dailies `/dailies/`, Today `/today/`
Post	Editor `/editor/`, Screen `/screening/`, VFX `/vfx/`, Music `/music/`, Post `/post/`
Release	Sales `/producer/#sales`, Deliver `/distribution/`, Fests `/festivals/`, Tax `/taxcredit/`
Always	Workflow `/workflow/`, Projects `/projects/`

One studio, five owners

A *production* is a named slot holding a snapshot of every module store — screenplay, boards, budget, schedule, cast, cut — plus a manifest of its photographs. The vault engine at `projects/lib-vault.js` creates them,

switches between them, and packs them as portable `.cinamate` archive files.

The Projects module also pushes a production to the **studio cloud**, and this is the fact that most changes how the platform feels: the cloud is shared, not personal. One catalogue serves the site and all five owners see all of it. A pushed production is listed to everyone alongside the owner who last saved it; anyone signed in can open it, and anyone signed in can remove it from the shared list — earlier copies are kept, and the confirmation prompt says so. Pushing under an existing name replaces the cloud copy. The transport takes 4 MB of written record per production and 48 MB of media in at most 48 parts, with a 900 KB ceiling per file: stills and short takes, not source masters. Anything over is refused whole, naming the file, rather than uploaded down to whatever happened to fit.

What stays on this device

Everything the platform saves lives in this browser first, under keys of the form `SB_<Module>_v1` in `localStorage` — sixty-odd across the twenty-eight modules — plus binary media in IndexedDB for wardrobe continuity photographs, scout photographs and cutting-room sources. The project index itself is `CIN_Projects_v1`. Two keys are deliberately excluded from every snapshot, archive and cloud push: `SB_LocalGPU_v1`, holding your render bridge address and its API key, and `SB_TMDB_v1`, holding a personal API key. They belong to the machine, not to the film, and nothing carrying a credential travels to another owner.

> **Clearing this browser's site data discards work.** A production not pushed to the studio cloud or exported to a `.cinamate` file exists nowhere else. A production opened on a second machine is a copy, and does not stay in step with the first until somebody pushes again.

Where to begin

Sign in and take the rail from the top. **Writer** is where a production acquires its first real content, and the dashboard tiles stay blank until it does. **Projects**, at the foot of the rail, is where a production is named, snapshotted, archived and shared.

Projects, The Vault & Cloud Sync

A production is not a document you open. It is the whole of this browser's storage, held under one name — and the Projects page is the only place that can move it, copy it, or take it away.

> **On this chapter.** Everything here was established by reading the platform's source rather than by operating it. The software is under active development; if a screen contradicts a paragraph below, the screen is what is running and this page is out of date.

What a production is made of

Two stores hold a film, and they behave differently.

The written record lives in this browser's localStorage, under keys matching `SB_<Module>_v<n>` — the pattern `KEY_RE` in `projects/lib-vault.js`. Script and scenes (`SB_Timeline_v1`), boards, budget (`SB_BudgetSheet_v1`), crew (`SB_Crew_v1`), schedule, clearances, the cut, and some seventy others. A key that does not match that pattern is not part of any production and the vault will not carry it.

The photographs live in IndexedDB, in three databases the vault names explicitly in `BLOB_DBS`: `cinamate_wardrobe` (continuity stills), `cinamate_scout` (location scouting), and `cinamate_cut` (cutting-room sources). Those bytes are never copied into localStorage. A production claims a photograph either because the record carries its project stamp, or because one of its `SB_*` stores still points at that id — which is how a scout photo, which carries no stamp at all, stays attributable to the right film.

The index of every production on the machine is `CIN_Projects_v1`: the active name, and a slot per project holding a full copy of that project's `SB_*` values plus a *manifest* of its media — ids and sizes, no bytes. Bytes stay in IndexedDB deliberately; five photographs written into the one record that indexes every project would exhaust the storage quota and take all of them down together.

Switching between productions

Open takes the outgoing project's live workspace, writes it into that project's slot, and only then rewrites localStorage from the incoming slot. The order matters: the snapshot is persisted before anything is cleared, so an interruption cannot leave a slot pointing at work that has already been thrown away. IndexedDB is not cleared on a switch — the other production's photographs stay on the machine, attributed to it by its manifest.

A name may be reused for nothing: starting a new project under an existing name is refused, and so is renaming onto one. Every module carries the active-project badge from `js/project-badge.js`. If another tab switches productions, open pages lock behind a reload prompt so two films cannot bleed into one another.

Backing up: the .cinamate export

Export backup writes a single file — `format: "cinamate/1"` — containing the project name, a timestamp, every portable `SB_*` value, the media bytes as data URLs, and `blobsMissing`: a list of files the production references that this machine could not produce, recorded at pack time so a later restore can tell "this backup never had them" from "this restore lost them". A cutting-room source above 16 MB is counted and reported, never read into the file.

Four things are deliberately absent.

NOT IN A BACKUP	WHY
`SB_LocalGPU_v1`	This workstation's render-bridge address and its API key. Machine-local, and a credential.
`SB_TMDB_v1`	A personal API key. Same reason.
`CIN_Learn_v1`	Cross-project learning. Outside the `SB_` prefix, so structurally excluded — it belongs to the machine, not to any one film, and it is neither backed up nor synced anywhere.
`CIN_Projects_v1`	The index of every project on this machine. Also outside the prefix, and not one production's to carry.

The two `SB_` exclusions are enforced twice — in the vault's `LOCAL_ONLY` pattern and again in the cloud function — because either side alone is one bug away from publishing a key to every owner.

> **A backup is readable by anyone holding it.** The file is plain JSON, not encrypted and not password-protected. It contains the crew directory as entered: names, rates, telephone numbers, email addresses and emergency contacts, alongside deals, budget actuals and everything else the production has recorded. Treat the file as you would a printed deal memo, and think before sending one over an ordinary channel.

Restoring

> **Restore is destructive.** Writing an archive removes *every* portable `SB_*` key in this browser and then writes what arrived. It is not a merge and there is no undo inside the workspace. The Projects page stashes the current workspace into its own slot before importing — that stash, or a backup file, is the only way back. An archive carrying no production data is refused outright rather than obeyed, because obeying it would clear the workspace and write nothing.

An archive from another owner or machine is treated as untrusted: every string is neutralised before a single key is written, with a short list of prose fields (script, notes, synopsis) kept exactly as typed. That happens in full before storage is touched, so a file that fails the check changes nothing.

The restore reports rather than reassures. It states how many media files were written, how many were unreadable and left out, how many this browser refused to store, and how many the production references but nobody has the bytes for. It claims to be complete only when all four of those are empty.

The studio cloud

The cloud is one shared namespace for five owner accounts, behind the owner login. This is the design, not an oversight: **every owner can list, pull, overwrite and delete every production**, including ones they did not

create. There is no per-owner partition and no private area. Pushing under a name that already exists replaces the cloud copy of that film, and the catalog records who saved it last.

Two protections exist, and they are narrow. A push may declare the version it believes it is replacing; if the cloud has moved on, the save is refused with a conflict rather than burying the newer one. And the badge holds back auto-sync when the cloud copy belongs to another owner and this tab has never taken it over. Neither prevents a deliberate Save to cloud from overwriting a colleague's work.

Versions: a ring of eight

Each push moves the copy it replaces into a rotating ring of eight superseded copies. History lists what is on file, with times and the owner who saved each; restoring puts one back, and the copy that was live at that moment is kept too, so the undo is not itself a way to lose work. A delete also lands in the ring, with a record of who deleted it and when.

> **The ring holds the written record, not the photographs.** Eight copies of every still would be a storage bill rather than a backup policy. Restoring an earlier version brings back the text of the production; it does not bring media back, and the response says so plainly rather than letting an operator infer otherwise.

> **Delete removes the media permanently.** Deleting a production from the cloud sweeps its media parts from the store. The written record stays recoverable through the ring; the photographs do not. This is deliberate and recent: the parts were previously left behind, which meant a production its owner believed deleted could still have its location interiors and wardrobe continuity fetched by anyone who had seen the name. That was a leak that looked like a safety net, so the sweep was made real and the deletion record now states that the media went. Export a `.cinamate` backup before deleting anything you may want the pictures of.

The four-minute auto-sync

Every module page runs a background sync on a four-minute interval (`SYNC_MS`), with a first pass around twenty-five seconds after load and one more attempt when the tab is hidden. It hashes the portable `SB_*` stores, does nothing if they have not changed, checks the cloud catalog, and pushes the active production under this owner's name.

It carries the written record only — never media. The server treats that silence as "this push is not about photographs" and carries the previous media state forward untouched, so the cloud copy's pictures remain whatever the last deliberate Save to cloud uploaded.

Working on two machines, take this literally: whichever machine has a Cinamate tab open is pushing its own idea of the production every four minutes. Pull before you start on the second machine, and close or reload the first — otherwise the version check will start refusing saves, or a quiet machine will push a stale record over a day's work. An archive approaching the cloud's four-megabyte ceiling is skipped by auto-sync entirely and becomes manual territory.

Limits that will stop you

LIMIT	VALUE	APPLIES TO
Written record	4 MB	One production's `SB_*` half in the cloud. Media travels separately and does not count towards it.
One media record	900 KB	Roughly three times a full-size scout JPEG. The cloud carries stills and short takes, not source masters.
One part	1 MB	A single upload request, sized to be cheap to retry.
Parts per production	48	Which is where the 48 MB per-production ceiling comes from — it holds by construction.

Anything over any of these is refused whole, before a single byte goes on the wire, with a message naming the offending file and its size, stating that nothing was sent, and offering the `.cinamate` export instead — which has no size limit. A production is never uploaded down to whatever happened to fit: an archive that arrives without its photographs must not look complete.

> **Not established from source.** The vault engine and the cloud function are exercised by `scripts/test_vault_blobs.mjs` against an in-memory media adapter, so the pinned behaviours above are the engine's. How a real browser behaves under storage pressure — quota refusals, an IndexedDB write aborted mid-restore — is reported by the page when it happens but cannot be characterised by reading, and no figure for it is offered here.

Workflow — Pipeline Mission Control

Every other module knows one thing about your production. The Workflow page is the only one that reads all of them at once, works out where the picture actually stands, and says what to do next. It writes almost nothing and decides nothing on its own — it is a mirror held up to the stores the rest of the platform has already filled.

> **On this text.** This chapter was written by reading `workflow/workflow.js`, `workflow/advisor.js`, `workflow/advisor-ui.js`, `workflow/index.html`, `workflow/workflow.css` and `js/learn.js`, together with the node suites that pin them. Nothing here was observed in a running browser. The platform is under active development; where this manual and the software disagree, the software is what runs.

Where the page gets its facts

The page is `workflow/index.html`. Its engine, `window.CWorkflow.assess()`, is a pure function: hand it a bag of already-parsed stores and it returns the whole assessment, with no reference to the document — which is why it can be executed in node, and is. In the browser, `gather()` fills that bag from `localStorage`, one `JSON.parse` per key, each wrapped so a corrupt value reads as `null` rather than breaking the page.

KEY	FEEDS
`SB_Timeline_v1`	project name, screenplay text, clips, characters, location bible
`SB_Writer_v1`	treatment beats and the Writer's project title
`SB_BudgetSheet_v1`	top-sheet categories, estimates, actuals, contingency
`SB_Budget_v1`	project mode, scale and the modelled incentive
`SB_ScheduleBoard_v1`	strips and the day each is boarded onto
`SB_ShootPlan_v1`	the first shoot date
`SB_LocalGPU_v1`	the Cinamate AI bridge URL
`SB_Captions_v1`, `SB_Credits_v1`, `SB_EPK_v1`	three of the four delivery steps
`SB_Cut_v1`	the Editor's last export, which satisfies the fourth
`SB_ReviewNotes_v1`	the dailies-notes count on the Review stage

`SB_Sales_v1`, `SB_Crew_v1` and `SB_Drafts_v1` are gathered as well but no stage consults them; the Advisor reads `SB_Crew_v1` for itself. Everything comes from this browser: the page makes no network call except one optional health ping to the bridge.

The seven stages

Completion is derived, never ticked off by hand. There is no "mark done" control on this page, and there is no half-done state — a stage is *Done*, *In progress* or *Up ahead*.

STAGE	MODULE	COUNTS AS DONE WHEN
1 Develop	Writer	the Studio's `scriptText` exceeds 200 characters, or the Writer holds at least one scene
2 Breakdown	Studio	the timeline holds at least one clip
3 Budget	Producer Suite	the top sheet's estimate subtotal is above zero
4 Schedule	Producer Suite	at least one strip carries a day of 0 or higher
5 Generate	Studio · Cinamate AI	there is at least one clip and every clip has a `videoUrl`
6 Review	Studio · Dailies	there is at least one clip and every clip's status is `approved`
7 Deliver	Tools	all four finishing chips are satisfied

The metrics under each card come from the same read. Budget prints the subtotal grossed up by `contingencyPct`, the project mode, and actuals if any. Schedule prints boarded strips against total strips, the number of distinct days, and the planner's first date as *Day 1*. Generate prints rendered against total, how many are rendering now, and the bridge URL or *Bridge not configured*. Deliver carries four chips — Captions, Credit roll, Press kit, Final export — the last satisfied either by every clip being approved or by the Editor having recorded a `lastExport`.

The header percentage is blunt arithmetic: finished stages divided by seven, rounded, so each stage is worth about fourteen points whatever it cost in real days. The *Next up* line names the first unfinished stage, quotes its hint and offers its button; when nothing is outstanding it reads *Pipeline complete — every stage is done*, followed by a clapperboard glyph. Only that first unfinished stage is promoted to *In progress*; a later stage with partial work of its own — clips rendering while the budget is still empty — also shows as in progress. The *Next up* line, not the badges, is the single answer to "where am I".

> **A budget built with the calculator can read as empty.** The Budget stage sums each line's stored `est` field directly. The Producer Suite also lets a line be written as Amount × Units × Rate, and those lines leave `est` at zero. The Advisor avoids this by reading through `CBudgetSheet.subtotal()` in `js/lib-money-sheet.js`, which honours the calculator; the pipeline stage and the Advisor's own *top sheet is empty* prompt do not. A sheet built entirely that way can carry a real total in the Producer Suite while this page still shows Budget unfinished and prompts you to seed it. Nothing is lost — only mis-reported.

The generation board and system row

Below the stages, every clip appears as a tile. Status is derived in one pass: `approved` or `generating` if the clip says so, *rendered* if it merely carries a video URL, *queued* otherwise. Tiles sort by urgency — generating, queued, rendered, approved — then by clip number, so what needs attention sits at the top left. Labels truncate at 26 characters; numbers pad to two digits.

The System row carries three chips: the bridge, the model (always reported as Cinamate AI, rendering on your machine), and the project type. When a bridge URL is configured the page fetches `/health` against it with a 2.5-second abort, once per URL, and rewrites the chip to say either that the bridge is online or that it is configured but not reachable right now. A failed ping is a reachability statement, nothing more.

The page re-renders on the browser's `storage` event — that is, when *another tab* saves — and whenever this tab becomes visible again. It does not poll. Work saved by other code in this same tab will not refresh it until you leave and return.

The Advisor

The Advisor is a separate layer: pure functions in `workflow/advisor.js`, mounted here by `workflow/advisor-ui.js`. It runs the script through `SBBudget.analyze()`, reads the budget through the shared sheet reader, and produces three things.

Prep actions

A ranked list of what is outstanding, each row a link into the module that fixes it. Four severities are printed as words, so the ranking never depends on colour:

SEVERITY	RAISED BY
NOW	no screenplay; script not broken down; permits at *Applied* or *Denied*; clearances still *Flagged* or *Clearing*
SOON	night scenes with no shoot-day plan; empty top sheet; no jurisdiction modelled; uncast roles; empty crew register; no insurance certificates; a cut exported with no delivery checklist
NEXT	unscouted locations; rendered clips awaiting approval; approved clips not yet on the Editor timeline
CLEAR	nothing outstanding

If there is no screenplay at all the list stops after one row — everything downstream is moot until there is a script.

Where to shoot it

The ranking scores every jurisdiction in the platform's incentive table on three things: the money back on this budget, weighted heaviest; how many of the script's look cues that place can supply, at fourteen points each; and a hand-authored crew-depth rating of one to three, at six points each. Look cues are drawn from the screenplay by keyword, plus a few implied by genre. Falling below a programme's minimum spend costs fifty points and exceeding its budget cap costs sixty — a penalty, not a veto, so an ineligible programme can still place if the money is large enough, and it is flagged in the reasons when it does. The top three are shown.

Two cautions the source states plainly. The incentive figures are headline programme terms, to be verified with a production accountant before anything is relied on. And when no budget has been seeded the ranking is modelled on a five-million-dollar placeholder — the heading says so in parentheses, and that parenthesis is the difference between a recommendation about your film and one about a hypothetical one.

Who to hire for this style

The staffing table starts from a core unit — eleven entries for scripted work, a five-person unit for documentary — and multiplies the elastic positions by scale read from `SB_Budget_v1`: 1 for indie, 1.5 for mid, 2.2 for studio. The

script's detected drivers then add specialists — stunt coordinator, marine coordinator and safety, on-set VFX supervisor, extras casting, animal wrangler, period set decorator. More than 30% night work adds lighting crew; more than eight speaking parts adds a 2nd AD. Each row states its reason.

Turning a recommendation into an action

Model this jurisdiction writes the chosen id into `SB_Budget_v1` as the `incentive` preference, so the Studio estimate and the Producer Suite immediately assume it. **Seed these as open positions in Crew** writes into `SB_Crew_v1`: one row per recommended head, named *(open position)*, with department, a union default per department, and every contact field blank.

> **Seeding crew rewrites the register.** The seeder skips any role name already on the books, so pressing it twice adds nothing — but it also will not top up a role you genuinely need more of. Rows are added to the front of the list, and there is no undo on this page. If `SB_Crew_v1` holds anything other than an array, that value is discarded and replaced.

What the self-learning paragraph is telling you

Beneath the staffing table sits a short four-line paragraph — the only place on this page that reports on the platform's own performance, and the one worth reading precisely.

Its first line is built in `js/learn.js` by `CLearn.learningReport()`, from the walk-forward record kept in `CIN_Learn_v1`. Before any closed budget line is folded into the learning store, the platform writes down what it *would* have predicted for that line from everything it knew at that moment, next to what the line really cost. The check compares those predictions against the raw estimates they replaced, over lines whose account already had at least two prior observations, and reports one of four things:

- **unproven** — fewer than five scored lines exist. The sentence says how many of the five are in hand and shows no percentage at all.
- **better** — calibrated estimates beat raw by a stated percentage, with the counts of lines that landed closer and further off.
- **worse** — calibration is making estimates a stated percentage *worse* than raw, and the sentence says to treat the seeded numbers as unhelped.
- **level** — the correction is not earning its place yet.

The second line is the gate. Nothing is learned from a production until it is marked wrapped in Projects, because a part-shot ledger reads as an enormous underrun and would drag every future estimate down. Until then this line tells you plainly that nothing is being learned.

The third line is the one to be careful with. It reports how many closed lines have been folded in, across how many accounts, and then an **average correction applied** — a multiplier such as ×1.95. That number is *not* an accuracy score, and the page labels it in full: how much correcting it has done, not how well it works. It rises as the platform gets things more wrong. A large correction average is not evidence that the learning is helping; only the first line answers that question.

The fourth line reports render speed from the same store: how many renders have been timed, the median wall-clock seconds per clip on this machine, and a trend of faster, steady or slower — computed only once at least eight renders exist. A count of cached research lookups follows if there are any.

> **The verdict is carried by the words, not the colour.** On screen the headline is tinted red for *worse* and green for *better*. In print there is no tint, and none is needed: the sentence states the verdict, the percentage and the sample size outright. Read the sentence.

What is pinned, and what is not

`scripts/test_workflow.mjs` executes the engine and fixes the stage rules above — 0% on an empty browser, Develop first, the rendered and approved ratios, the contingency arithmetic, the distinct-day count, the Editor export satisfying the final delivery step, 100% with nothing next, and that malformed stores never throw. `scripts/test_advisor.mjs` runs the Advisor against the real incentive table and pins the look extraction, the ranking penalties, the driver-led staffing additions and every prep-action rule. The self-learning paragraph is pinned by a further suite that renders the panel and asserts no code in it reads the old correction average as though it were accuracy.

What no test can establish, and this manual does not claim: whether the incentive terms are current, whether a crew-depth rating reflects today's market, or whether the bridge on your machine is healthy. Those are questions for a production accountant, a line producer and a ping.

The Writer — Treatment to Screenplay

The Writer takes a treatment — a PDF from a producer, a Word document, a paragraph pasted out of an email — turns it into scene beats you can reorder, retitle and rewrite, and emits a Fountain draft the Studio can break down. It is the front door of the platform, and the format it emits is the format every department downstream inherits.

> **How this chapter was written.** By reading `writer/index.html`, `writer/writer.js`, `writer/lib-treatment.js`, `js/lib-scenes.js` and `scripts/test_writer.mjs`, and by running the two engines together outside the browser to confirm what one hands the other. Cinamate is under active development; where this manual and the software disagree, the software is what runs.

What it accepts

The module is `writer/index.html`, served at `/writer/` and linked from the dashboard rail. Panel 1, **Treatment in**, offers a drop zone, a **Choose file** picker and a collapsed …*or paste the treatment text* box. Five extensions are handled, and the picker and the code agree on them: `.pdf`, `.docx`, `.txt`, `.md` and `.fountain`. Anything else is refused with *Use .pdf, .docx, .txt, .md or .fountain*. The paste route requires at least 40 characters.

A PDF is read with the vendored pdf.js at `static/vendor/pdf.min.js`: text items are grouped into lines by vertical position, a gap over twenty units taken as a paragraph break. That is a reconstruction of the page, not a transcript, and a scanned PDF with no text layer yields nothing. A DOCX is unzipped with JSZip and `word/document.xml` read directly — only `<w:t>` runs survive, and tracked changes, comments and tables are ignored; a zip without that part is refused as *Not a Word document*. The text formats are taken as-is. Whatever the route, the result passes through `cleanText`, which drops any line that is only a page number (*Page 3 of 12* included), rejoins words hyphenated across a line break, and re-flows hard-wrapped lines into paragraphs. A blank line is the only paragraph break it trusts.

What the Writer thinks a scene is

There are no acts, sequences or beat sheets as objects anywhere in `writer/lib-treatment.js`. The single structural unit is the scene beat, and a beat starts wherever `isHeadingLine` says a heading starts. That function accepts a line of 90 characters or fewer that is a slugline; or begins `ACT`, `PART`, `CHAPTER`, `SEQUENCE`, `SCENE` or `BEAT`; or is a Markdown `#` heading; or is numbered, as in *12. THE HEIST*; or is simply in capitals and not a run-on sentence. Everything else becomes the body of the current beat, and prose before any heading becomes one beat per paragraph. Ahead of the first beat, an opening block under 80 characters is lifted as the **Title**, a line reading *by* … as the **Writer**, and *Logline:* … as the **Logline**; those, plus a draft date, stay editable in panel 1.

Each beat is then given a slugline by `guessSlug`. A heading that is already a slugline is kept, upper-cased, its dashes normalised, and a time of day appended if it has none. Otherwise the interior/exterior and the time of day are inferred from word lists scanned in the body — *kitchen* and *cockpit* lean interior, *street* and *rooftop* exterior —

and the location is the heading with its `ACT`, `SCENE` or numbering prefix stripped, truncated to 48 characters, falling back to a location cue from the body or the literal word `LOCATION`.

It is worth seeing that inference miss. The treatment fixture in `scripts/test_writer.mjs` contains the structural heading *ACT ONE*, which becomes the slugline `INT. ONE - DAY`; a beat titled *THE CALL* becomes `EXT. THE CALL - NIGHT`; *SEQUENCE 4* and *PART III* both become `EXT. LOCATION - DAY`. These are legal sluglines the rest of the platform will treat as real places, so a location bible built off an unedited import contains sets called ONE and LOCATION. Panel 2 exists so you fix them before they travel.

Character chips come from `extractCharacters`: capitalised names count double, repeated Title Case words once, a stop-word list removes *THE*, *FADE* and their kin, and a name needs two mentions to survive. It reads the heading as well as the body, which is why the shipped fixture yields MARA alongside ONE, CALL and HIGHWAY. Treat the chips as a suggestion; casting is not built from them.

The scene cards

Panel 2 is the whole structural toolkit. Each card carries its position number, an editable slugline, an editable body and its character chips; ▲ and ▼ reorder, ✕ deletes, **+ Scene** appends a blank card seeded `INT. NEW SCENE - DAY`, and **New project** starts over. Editing a body re-derives that card's characters. Editing a slugline does *not* re-run `guessSlug` — what you type is what ships, with no validation. That matters, and the next section explains why.

> **Both ✕ and New project discard work immediately.** ✕ deletes a card with no confirmation and no undo. **New project** asks once — *Start a new project? The current scenes are discarded.* — and on *OK* clears the title, writer, date, logline and every scene from this browser. Export the `.fountain` first if there is anything in it you want.

The screenplay it emits, and whether the Timeline reads it

Panel 3 previews the Fountain draft live and offers **Copy**, **⬇ .fountain** and **→ Send to Studio**. The shape is fixed:

```
Title: THE LAST DISPATCH
Credit: Written by
Author: Kyle Francis
Draft date: 2026-08-20
Source: Treatment developed in CINAMATE Writer

= A night-shift dispatcher fields a call from her own future.

INT. ONE - DAY

MARA (30s), tired eyes, works the graveyard shift...

EXT. THE CALL - NIGHT

A voice she recognizes — her own — warns her about tomorrow night.

MARA
Who is this?
```

Dialogue is the one thing carried through from treatment prose: a paragraph of the form *MARA: "Who is this?"* becomes a cue line and a speech, whether it stands alone or sits mid-paragraph, and the surrounding action is split around it. There are no transitions, no parentheticals, no dual dialogue and no scene numbers.

Does it parse? Yes, for anything the Writer generates itself. Every slugline `guessSlug` emits opens with `INT.`, `EXT.` or `INT./EXT.`, followed by a non-empty location and a `- DAY`/`- NIGHT` tail. The scene model in `js/lib-scenes.js` accepts all three tokens, requires a separator after them and a location that is not empty, and both conditions are always met — `guessSlug` falls back to the literal word `LOCATION` rather than emit a headless slug. Running the two modules together over the shipped fixture, three cards produce three scenes numbered 1, 2 and 3, with location, time of day and interior/exterior read correctly on each, and the MARA cue recognised as dialogue by `CScenes.speaksIn`.

On the preamble. The Writer emits no `FADE IN:` at all. What precedes the first slugline is the Fountain title page and the logline, and `js/lib-scenes.js` holds that separately as the preamble at ordinal 0 — never as a scene, never dropped. This is the exact defect that file was written to end: seven of the eight scene splitters it replaced counted a preamble as scene 0 and then numbered the first real scene 2. A Writer draft opens on scene 1.

Where it diverges, and this is the paragraph to read twice. The slugline field on a card is free text, and a card whose slugline does not carry an `INT`/`EXT`/`EST` token is not a scene break downstream — its content is silently absorbed into the previous scene. Five cards edited as below produce three scenes in the Timeline, and neither end warns you:

CARD SLUGLINE IN THE WRITER	WHAT THE TIMELINE MAKES OF IT
`INT. KITCHEN - DAY`	Scene 1 — KITCHEN, DAY
`Kitchen, night`	Not a scene. Body folded into scene 1
`THE ROOFTOP`	Not a scene. Body folded into scene 1
`4A INT. STUDY - NIGHT`	Scene 4A — STUDY, NIGHT
`INT. NEW SCENE - DAY`	Scene 3 — NEW SCENE, DAY

Two rules follow. Keep the interior/exterior token on every card you rename: the card count in panel 2 and the scene count in the Studio should always match, and when they do not, a slugline is the reason. And note the fourth row — a printed number typed into a card *is* honoured, and it flips the whole script to numbered, at which point unnumbered cards fall back to their position. Number every card or none. Left alone, a Writer draft carries no numbers at all, so the scene label on the call sheet, the slate and the daily report is the scene's position in the file: right for a first draft, and not a shooting script's numbering.

> **The Writer is not a screenplay importer.** `.fountain` is in the file picker, but the engine reads everything as a treatment. Feeding it a finished script was checked and does real damage: a character cue standing alone on its line is a capitalised line, so it is read as a heading and becomes a scene — `EXT. HANK - DAY` — with the speech below it demoted to action. On a numbered shooting script, *24 INT. KITCHEN - DAY* comes out as `EXT. 24 INT. KITCHEN - DAY - DAY`: the printed number 24 survives only as part of the location, the interior/exterior is inverted, and scene 24 becomes scene 1. To bring an existing screenplay into Cinamate, use the Studio's own import, covered in the Timeline chapter. The Writer is for material that is not a screenplay yet.

> **Nothing in the test suite defends this agreement.** `scripts/test_writer.mjs` runs with `npm test` and checks the Fountain string with its own regular expressions; no suite in `scripts/` loads `writer/lib-treatment.js` and `js/lib-scenes.js` in the same process. The behaviour described above was established by doing so by hand, and it is not pinned against future change.

Export and the hand-off

Copy puts the Fountain text on the clipboard, and advises the download if the browser blocks it. **⬇ .fountain** saves a file named from the title, lower-cased and hyphenated, or `cinamate-draft.fountain` if there is none; Fountain is plain text, so it opens in a screenwriting application as well as in Cinamate. **→ Send to Studio** reads the Studio's store `SB_Timeline_v1`, replaces its `scriptText` with the Fountain draft and its `projectName` with the title, writes it back, and navigates to `/timeline/` after a short pause. If the write fails it reports *Could not save — storage full?* and stays put.

> **Send to Studio overwrites the Studio's screenplay of record, without asking.** The Studio's clips, parse result and location bible are left untouched, so immediately after a send they describe a script that is no longer there. Re-parse in the Studio before reading anything off the timeline.

What is stored, and where

The Writer keeps everything in `SB_Writer_v1` in this browser's local storage, written on every keystroke: the project fields, the scene array — slugline, body and characters per scene — and the name of the imported file. No scene numbers, because a scene's number is its position. Nothing is sent to a server and nothing follows you to another browser or machine, so the `.fountain` download is the portable copy.

Three other places read that key. The dashboard counts `SB_Writer_v1.scenes` for its scene tile. `boards/lib-shots.js` seeds the shot boards from the Studio's clips when there are any and from the Writer's beats when there are not, so a treatment that never reached the Studio can still be storyboarded. The Workflow advisor reads it as one input among several.

The page and runtime figures

Panel 1 reports scenes, words, an estimated page count and an estimated runtime. The arithmetic is explicit in the source: 450 words is one treatment page, a treatment page expands to 3.5 screenplay pages, and one screenplay page is one minute — so the page and minute figures are always the same number. On the shipped fixture, 43 words of treatment estimate 4 pages and 4 minutes, while the scene model measures the actual Fountain output at three eighths, or one page. They answer different questions: the Writer forecasts the screenplay this treatment would become, the Timeline measures the text that exists today. The schedule and the board are built from the second.

The Timeline I — Script Import & Breakdown

The Timeline is where a screenplay stops being a document and becomes production data: a numbered scene list, a cast, a set list and a page measure. This chapter covers getting the script in and broken down. Estimating and exporting it is the chapter that follows.

> **On this text.** Every statement here was established by reading the source — `timeline/parser.js`, `js/lib-scenes.js`, `timeline/timeline.js` and the two node suites that pin them. The platform is under active development. Where this manual and the software disagree, the software is what runs.

Getting the screenplay in

The page is `timeline/index.html`. Three routes lead into it, and all three end in the same parse.

- **Import** in the top bar, **Import file** on the yellow script bar and **Import Script** in the empty-state panel open the same picker, which accepts `.txt`, `.pdf` and `.fdx`. Anything else is refused with *Unsupported file — use .txt, .fdx, or .pdf.*
- **Paste script**, or the gold **Script** button, opens the screenplay editor: one textarea, which is the screenplay of record. The timeline is derived from it, never the other way round.
- **Re-parse** re-runs the parse over whatever that textarea currently holds.

A `.fdx` is read as XML and only six paragraph types survive: Scene Heading, Action, Character, Dialogue, Parenthetical and Transition. Anything Final Draft calls a Shot, a General element or a note is dropped before the parser sees it. A `.pdf` is read with the vendored pdf.js at `static/vendor/pdf.min.js`: text items are sorted top to bottom then left to right and bucketed into lines within a vertical tolerance of five units. That is a reconstruction, not a transcript, and a PDF with no text layer yields nothing at all.

Because pasted and extracted text arrives with its line breaks destroyed, `normalizeScriptText()` runs on every route. Text counts as flattened when it is one or two lines carrying hundreds of characters, when its average line runs past 130 characters, or when long lines dominate. The unflattener then rebuilds breaks before sluglines, character cues, parentheticals and transitions. **Unflatten** in the editor runs that pass on demand and reports the line count before and after.

> **Import replaces the timeline.** Importing a file parses immediately and overwrites the clips, the parse result and the location bible with no confirmation. **Re-parse** asks first, warning that clips carrying generated video will be rebuilt. **+ New script** asks first, then clears the screenplay, the timeline, the characters and the parse result. Undo restores the previous state within the session only — the history is held in memory and does not survive a reload.

What counts as a scene heading

One grammar answers this for the whole platform: `CScenes.parseSlug()` in `js/lib-scenes.js`. The Timeline delegates to it and keeps no second opinion. A heading opens with an interior/exterior token — `INT`, `EXT`, `EST`, or a

two-sided form (`INT/EXT`, `EXT/INT`, `I/E`, `E/I`, with or without spaces and full stops) — followed by a space, dash, colon or full stop, and then somewhere to be. That separator requirement is what keeps *INTERCUT*, *EXTRAS fill the room* and *Interior designers argue* out of the scene list. A bare `INT.` with no location is not a scene.

LINE	READ AS
INT. KITCHEN - DAY	INT · KITCHEN · DAY
INT KITCHEN - DAY	accepted; the full stop is optional
EST. THE CAPITOL - DAY	EST · THE CAPITOL · DAY
I / E PATROL CAR - DUSK	INT/EXT · PATROL CAR · DUSK
INT. PARIS - LEFT BANK - NIGHT	location keeps both halves
24 INT. FARMHOUSE KITCHEN - NIGHT 24	number 24; right margin stripped
MONTAGE - THE TRAINING	a scene on this page only
1. KITCHEN - DAY	a scene on this page only; number 1

The last two rows matter. `timeline/parser.js` layers three heading forms of its own on top of the shared grammar, because a treatment or a documentary transcript prints no INT./EXT.: *SCENE 12 - KITCHEN*; the all-caps beats FLASHBACK, FLASH CUT, MONTAGE, DREAM, INTERCUT, BACK TO, LATER, TIME CUT and SERIES OF SHOTS; and the numbered form *1. KITCHEN - DAY*. These break scenes here and nowhere else on the platform. A beat heading has no location of its own, so *MONTAGE - THE TRAINING* produces a location row literally named MONTAGE, and a flashback heading one named FLASHBACK. Expect to rename or delete those.

For flattened text the parser keeps one unanchored copy of the interior/exterior token, since the shared expression is anchored to the start of a line and a PDF paste runs three sluglines into one. `scripts/test_parser_slug_agreement.mjs` stops that copy drifting: over 180 assertions require that what the shared model accepts is also found mid-line here, with the same number, location, time of day and interior/exterior value, and that what it rejects is rejected here too. Run it with `node`; it either passes or it names the disagreement.

Scene numbers, and the A-scene

A scene number is not a number. *4A* is a scene number; so is *A4*. Each scene therefore carries six identity fields, and the right one depends on what you are doing with it.

FIELD	MEANING	FOR 4A
`ord`	position in the file; preamble is 0	3
`number`	the script's own printed number, verbatim	4A
`n`	numeric, for arithmetic and filters	4
`label`	what a human reads, what goes on a call sheet	4A
`key`	stable identity, so 4 and 4A cannot collide	4A
`sortKey`	collation only: 4 < 4A < 4B < 5	4.01

The printed number always wins; the ordinal fills in only where the script printed none. Where a shooting script prints its numbers down both margins, the right-hand copy is stripped before anything reads the line, and only when it matches the number on the left, so a location genuinely ending in a numeral survives. A bare number on its own line above a heading — how PDF extraction routinely delivers it — is attached to the heading below, across blank lines. `SC` and `SCENE` prefixes are accepted and stripped.

Two consequences to hold on to. A `FADE IN:` preamble is captured separately and is not a scene, so the first real scene is scene 1, not scene 2. And on a numbered script, asking for scene 3 when the printed numbers run 1, 2, 4A returns nothing rather than the third scene in the file — the platform will not invent a scene the production does not have. `scripts/test_scenes.mjs` pins this with over 150 assertions, including that 4A is a real scene break and that its content is not swallowed into the scene above.

Time of day, and where the parser stops being certain

Time of day is taken from the tail of the heading after the last dash, and only when that tail begins with a recognised word: CONTINUOUS, MOMENTS LATER, LATER, SAME TIME, SAME, MORNING, AFTERNOON, EVENING, MIDNIGHT, PRE-DAWN, DAWN, DUSK, SUNSET, SUNRISE, NIGHT, DAY, MAGIC HOUR. Anything else in that position stays part of the location. CONTINUOUS and LATER are carried through verbatim rather than resolved to a clock time, which is correct — they are relative, and only the script supervisor can settle them.

> **Three cases that come back less certain than they look.** A heading with no time of day leaves the scene's time of day empty — but the clip built from it is stamped `Day`, because the inference falls back to Day when it finds no NIGHT, DAWN or DUSK: the scene says unknown, the clip says Day. Second, *INT. KITCHEN - NEXT DAY* does not register, because NEXT DAY is not a recognised time word: the time of day comes back empty and the location becomes *KITCHEN - NEXT DAY*. Third, *INT. KITCHEN - DAY (FLASHBACK)* parses with a time of day of *DAY (FLASHBACK)* — the flashback rides inside that field rather than being flagged. Confirm all three against the script before anything is scheduled from them.

The platform does model this uncertainty, in `js/lib-storyday.js`, which marks every story-day boundary CERTAIN or UNCERTAIN. CONTINUOUS, SAME TIME, LATER, an explicit *next day* cue and a NIGHT-to-DAY transition are certain; DAY to DAY, NIGHT to NIGHT, DAY to NIGHT, a flashback and any heading with no printed time are guesses, defaulted to the same story day and reported as guesses. Note the scope honestly: of the shipped pages only `wardrobe/index.html` loads that module. The Timeline page does not, and shows no story-day column.

The page measure: eighths

A screenplay page is 55 lines, so one eighth is 55/8 — about 6.875 lines. The measure counts blank lines, because a blank line occupies the page, and counts a long line as the rows it would occupy when wrapped at the standard element widths, 60 columns for action and 35 for dialogue. A scene is measured as its heading plus its body, rounded, with a floor of one: a scene that exists occupies at least an eighth on the board. Page count is the eighths total divided by eight, to one decimal place. The constant is anchored by test, not assumed — 55 lines of content measures as one page, within an eighth.

> **Two measures exist.** The Producer's Estimate panel on this same page does not use the measure above. It carries its own splitter, sizes a scene at roughly five content lines to the eighth with blank lines discarded, and requires the full stop after INT/EXT — so *INT KITCHEN - DAY, EST.* headings and prefixed A-scenes are invisible to it. Its page count will not always match the parser's.

From scenes to clips

The Timeline's visible unit is not the scene but the clip — one previsualised beat. One clip is emitted per shot, and a shot is either a dialogue block or a group of sentences from an action paragraph, gathered until the group reaches six words. Each clip carries its scene index, the scene heading verbatim, the location and time of day lifted from that heading, a shot type and camera move inferred by keyword, and a rotating label from a fixed list (Opening scene, Character intro, Dialogue, Action beat, and so on). The labels are decoration; the heading is load-bearing.

Where the parser finds no heading at all it does not fail: it opens a single fallback scene called `SCENE 1` and files every shot into it. A feature that failed to unflatten therefore appears as one enormous scene with a hundred clips, not as zero scenes.

> **The quality warning is not a safety net.** `parseQualityWarning()` holds two messages — no scene headings found, and many shots in a fallback SCENE 1 — but they require four and six fallback scenes respectively, and the parser creates at most one fallback scene per run. Neither can fire from a normal parse. Judge a parse by the clip count and by the Characters and Locations panels, not by the absence of a warning.

Characters

Names are gathered in several passes over the normalised text. A character cue is an all-caps line of 2 to 40 characters that is not a heading, not a transition, not a parenthetical and does not end in punctuation. Dual dialogue survives in both its forms — the Fountain caret, and the two columns a printed page merges onto one line — so a dual scene does not lose one actor's day. Fountain's `@NAME` and inline *NAME: dialogue* are read as cues. An action-line introduction registers when the parenthetical after the name is descriptive rather than a delivery note: *MERCER (50s, silver hair, ex-military)* yields both the name and the description, while *(V.O.)*, *(CONT'D)* and *(whispering)* yield nothing. Titled names (Mr, Dr, Det, Agent, Sgt, Officer, Captain) are picked up separately, and background roles such as FLIGHT ATTENDANT come in through a looser gate. Against all of that, a stop-list rejects the words that look like names and are not: INT, EXT, FADE, CUT, the times of day, the common location nouns, weather. The result is deliberately conservative and will miss people; **+ Add Character** is how you put them back.

Each character holds a description, body type, wardrobe, voice profile, default emotion, cast role (lead, supporting, background, crowd, voice_only), face lock, lip-sync and a reference image. Role is inferred — a one-word name with dialogue reads as a lead, a multi-word name without dialogue as background. Typing into Description or Wardrobe marks that field as yours and the enricher will not overwrite it. **Sync from parse** re-runs the local extraction; signed in, it also requests an appearance-only character bible from the enrichment agent, falling back to the local extract when that is unavailable. Parsing itself needs no network.

> **Deleting a character is not local to the list.** The control asks for confirmation, then removes that character from the list and from every clip that referenced them.

Locations

A location's key is its heading location, upper-cased with whitespace collapsed, so the INT. and EXT. of one set share an entry. The bible is assembled from three sources in order — the clips, the parsed scenes, and the screenplay text, the last of which also honours explicit `Location:` lines. Each entry carries a name, a description seeded from the heading, the indices of the clips that play there, a lock flag, a reference plate and a consistency phrase injected into the prompt when the location is locked.

> **Sync from clips rebuilds the bible from nothing.** Both a fresh import and the Locations panel's sync button empty the location bible and reconstruct it. Rebuilt entries are unlocked, with no plate and no consistency phrase, so locks, uploaded plates and phrases entered by hand are not carried across. Lock a location after the breakdown is settled, not before.

Editing, splitting, merging and reordering

The screenplay text is the record, so the honest answer to "how do I split a scene" is: add the slugline in the script editor and re-parse. To merge two scenes, delete the second slugline and re-parse. There is no scene-level split, merge or renumber command on this page. Scene identity comes from what the script prints, which is the behaviour you want when the call sheet, the slate and the sides all have to agree.

At clip level the controls are **+ Clip**, which appends an empty beat; **Duplicate clip**, which copies the selected clip as a draft with its video link cleared; drag and drop on the clip row to reorder; and **Undo** / **Redo**, holding up to fifty steps for the session. Reordering renumbers every clip and reassigns its identifier from the new position, but does not renumber scenes: each clip keeps the heading it was built from. There is no delete-clip control — to remove material, edit the screenplay and re-parse.

The right-hand inspector edits one clip: its scene description, its emotion, and three groups of prompt parameters — Scene and setting (location, time, weather, season), Camera (angle, film grade, colour, saturation) and Atmosphere (lighting, mood, FX, sound), each with a toggle deciding whether it reaches the prompt. Editing here changes the clip alone: it does not write back to the screenplay, and the next re-parse overwrites it.

What is stored, and under which key

Everything on this page lives in this browser's local storage. None of it is a server-side record, and clearing site data clears the breakdown.

KEY	HOLDS
`SB_Timeline_v1`	the project: clips, characters, location bible, global settings, assembly, parse result, project name, screenplay text
`SB_Editor_embed_v1`	the embedded editor's own timeline state
`SB_LocalGPU_v1`	local generation bridge URL and key, this browser only
`SB_Timeline_script_hint_v2`	a flag marking the one-time hint as shown
`SB_Scenes_v1`	the shared scene store defined by `CScenes`

One honest note on that table. `SB_Scenes_v1` is the canonical scene register — its shape, its helpers and its key are all defined and tested — but no shipped page was found to write it. The department modules that need scenes call the shared splitter on their own copy of the script text instead.

Save project writes the whole state out as a file, screenplay and parse result included, and **Load project** reads one back. That is the only copy of a breakdown that survives a cleared browser; take one before re-parsing a script that has had a day of hand work put into it.

The Timeline II — Estimation & Exports

A broken-down script is a list of obligations. This chapter is about turning that list into a number a financier will read, and then getting the work back out in a shape another machine will accept. Both halves are arithmetic on what Chapter 05 produced; neither invents anything the script did not already say.

> **How this chapter was written.** Every statement below was established by reading the source files it names — no part of the platform was operated to produce it. Development continues, and if the software in front of you says something different from this page, believe the software.

Two estimates in one panel

The *Producer's Estimate* panel on the Timeline page (`timeline/index.html`, the `budgetPanel` disclosure) is drawn by `timeline/timeline-budget.js`, which exposes itself as `SBBudget`. It answers two questions from the same parsed script: what the AI rough-draft preview will bill and how long that run takes, and what the script would cost to shoot for real as a tiered line-item budget with a shoot-day schedule attached. Tier selections persist under the storage key `SB_Budget_v1`, and the panel recomputes only when the script, clips, characters, locations, model, quality or clip duration changed.

The measurement pass

`analyze()` runs first and everything downstream depends on it. It prefers `scriptText` when that is longer than 200 characters, otherwise it reassembles the text from the clips' headings, descriptions and dialogue.

Pages come from eighths. The text is split at sluglines (`INT.`, `EXT.`, `INT/EXT.`, `I/E.`) and each scene sized at five content lines to the eighth — the constant is `LINES_PER_EIGHTH = 5`. Pages is the sum of eighths over eight. The fallbacks matter: with fewer than two sluglines, page count reverts to a word count at roughly 200 words per page, and with barely any text, to clip seconds over sixty. Runtime in minutes is set equal to page count — the one-page-one-minute rule of thumb, stated as such.

Scenes and locations are read off the headings: each is classified interior or exterior, tested for `NIGHT`/`DUSK`/`EVENING`/`MIDNIGHT`/`PRE-DAWN`, and reduced to a location name. Unique locations is the larger of that set and your location bible. **Cast is ranked, not counted** — characters are ordered by how many clips they appear in, the top one or two become leads, the next six at most supporting, everything beyond that a day player. A script with forty speaking parts still models two leads and six supporting roles.

Budget drivers are keyword families, twelve of them, each a regular expression counted over the whole script and weighted out of ten: fire and explosions and VFX/creatures at 9; stunts, water work and period setting at 8; vehicle action 7; weapons and crowds 6; animals 5; weather and child actors 4; aerial work 3. The *complexity* figure shown as `n/100` caps each driver at ten hits and normalises against 72, the maximum weight sum. Genre is inferred the same way from seven keyword families, with Drama as the floor and Action taking over if the physical-action drivers outscore it.

> **These are word matches, not comprehension.** A character named Hunter, dialogue about a storm that never appears on screen, or a metaphorical "explosion of laughter" all count. Read the driver chips before you trust the schedule they inflated.

The schedule the money hangs on

Shoot days are computed before any dollar figure, and almost every line item is a function of them. The pace is the production scale's pages-per-day — 7 at micro-budget, 5.5 low, 4.5 indie, 3.5 mid-level, 3 studio, 2.2 tentpole — divided into the exact eighths measurement, then multiplied by a *driver load*: one, plus fifteen per cent of the night-scene fraction, plus small increments for stunts, pyro, water and crowds, plus a flat ten per cent when the VFX tier is heavy or above. The floor is five days. Prep is 1.1 times the shoot in weeks (1.6 on crews of 110 or more, minimum two); post is 2.4 times the shoot plus eight weeks for moderate VFX or sixteen for heavy, minimum eight. The week is five days unless you say otherwise; six- and seven-day premiums come from `TMoney.TC_DEFAULTS` in `tools/lib-money.js`, and are reported unavailable rather than quietly applied as 1× when that file is absent.

Cast weeks, drops and pick-ups

Supporting cast are paid for the weeks they are *held*, not the days they work. `castWeeks()` turns the worked shoot days into billed weeks, holds and drop/pick-up arithmetic. The defaults in `CAST_RULES` — five days per week, a minimum ten free days before a drop is permitted at all, one drop per performer, one day of written notice, a one-week minimum guarantee restarting on re-engagement, and `asOfDay: null` meaning nothing has been shot yet — are all parameters, because they are contract terms. A drop whose notice date is already past is *refused* into `refusedDrops` with its reason, and its idle days stay billed as holds rather than the panel quoting a saving nobody can take.

The tiers, and what each assumes

SELECTOR	DEFAULT	WHAT CHOOSING IT CHANGES
Production scale	Indie ($2M–$10M)	Pages per day, crew size and day rate, art allowance, script rights, producers, sound, music, DI, editorial, casting
Director tier	Emerging	One flat fee range, first-time through legend
Star / lead tier	Rising talent	Flat run-of-picture fee × number of leads
Supporting cast	Scale / indie	The per-role band the weekly cost is clamped into
Crew / unions	Non-union	Fringe rate (28 / 32 / 40 %), a labour multiplier, the performer day and week rates
Locations	Local practical	Per-day fee, per-location permit and restore, travel percentage
Camera & equipment	Indie kit	A weekly rental band, over shoot weeks plus one prep week
VFX intensity	Auto	Shot count and per-shot cost; auto derives both from the VFX and pyro drivers
Genre	Auto	Which benchmark the total is compared against — nothing in the budget itself
Tax incentive	None	Estimated recovery and net cost; twenty-one jurisdictions modelled

Leads are dealt as flat fees because that is how they are actually dealt. Supporting roles are not: their weekly rate — $1,800 non-union, $2,812 on a SAG Low Budget Agreement, $4,326 on SAG Basic — is multiplied by the span weeks the day-out-of-days produced, then clamped into the chosen band. Day players are the day rate ($400 / $810 / $1,246) times worked days.

Lines are grouped Above the line, Production, Post-production and Other on the Movie Magic-style account skeleton, with crew labour, equipment and the art allowance split across department accounts by fixed percentages. Insurance is 2.5 % of direct costs, legal and finance 1.5 %, a completion bond a further 2.5 % on indie-financed scales only, general expenses 4 % of below-the-line, contingency a flat 10 %. The account number prefix on each label is load-bearing — the Producer Suite routes a seeded line by matching it. The headline is three numbers: low, high, and a "likely" that is simply the midpoint. Incentive recovery is total × qualifying-spend fraction × credit rate, with a warning when the estimate falls below a programme's minimum spend or above its cap.

How a producer corrects the model

Inside the panel the corrections are the ten selectors above, plus the retake multiplier and parallelism for the AI figure. The engine accepts more than the panel offers: per-scene breakdown tags (`unitOverrides`) replace the keyword-derived counts for extras, stunt, pyro, water and animal days; real day assignments from the stripboard (`castDood`) replace the script-order day-out-of-days entirely; `castRules` and `daysPerWeek` replace the default agreement terms. Those arrive from the Producer Suite, and the board's numbers win.

Documentary mode

The panel carries a Feature Film / Documentary switch, and detection is automatic enough to prompt: when interview, archival and narration cues score 60 or more, a scripted estimate offers to switch. Documentary numbers come from `timeline/timeline-doc.js` (`SBDoc`), which returns the same groups-and-totals contract so the rest of the panel works unchanged. Nothing of the scripted model carries over. There are no cast tiers and no stunt units; instead four scales from DIY to premium/streamer, an archival appetite priced in finished minutes at an all-media rate per minute, a music posture, a crew basis, and a shoot of interview days plus vérité days plus one travel day per region. Edit weeks come from weeks-per-finished-hour bands, and a grants panel lists funding offsets. Incentive eligibility is adjusted where documentary rules are known — New York excludes documentaries outright, Georgia gives the 20 % base without the logo uplift.

> **The switch writes outside this module.** Flipping to Documentary also rewrites the `strategy` field inside the `SB_Sales_v1` storage key so the Sales tab follows, and flipping back resets a documentary strategy to `indie`. Any sales strategy you had chosen by hand is overwritten without a prompt.

An estimate is a model, not a quote

Say this to anyone you hand the panel to. The tables are calibrated against published 2025–26 union scale, permit schedules, rental price lists and trade-press deal reporting, and every figure is sourced by line in `docs/PRODUCTION_PRICING.md`; the documentary constants the same way in `docs/DOCUMENTARY_MODE.md`. Read them before you defend a number. Both publish their own limits, and these bite hardest: marketing and print costs are excluded entirely; fees above "established" are reported figures, not scale; the day-out-of-days schedules scenes in script order with no stripboard optimisation, so supporting spans run conservative; genre benchmarks are inflation-adjusted 2005-vintage data, context rather than forecast; and incentive recovery is modelled on headline terms, where the real number is a production accountant's opinion letter.

The preview half of the panel carries the same warning: clip count times average duration times the model's dollars-per-second, multiplied by a retake factor of 1.6 — the average generations per kept clip — with stills at four cents each. A local-render build prices at zero, because there the cost is electricity.

Getting the work back out

`timeline/timeline-export.js` (`SBExport`) produces four things, and only four.

ARTIFACT	FILE	WHAT A RECEIVING APPLICATION DOES WITH IT
CMX-style EDL	`cinamate-timeline.edl`	An online bay, finishing house or NLE conforms the cut: each event carries a source in/out and a record in/out, and the reel names say what media to relink
Project JSON	`cinamate-timeline-project.json`	Cinamate reads it back — clips, characters, location bible, globals, assembly, project name, script text and parse result
Stitched master	`cinamate-final.mp4`	A finished single-file cut of the approved clips, assembled in the browser
Clip archive	`cinamate-final.zip`	The fallback when stitching is unavailable: each clip as `clip_NN.mp4`

The EDL opens `TITLE: CINAMATE TIMELINE`, then `FCM: NON-DROP FRAME`, then a comment stating the frame rate the numbers were counted in — an EDL carries no rate of its own beyond the FCM line, so a conform at the wrong rate is otherwise silent. Events are numbered from 001 and butt together, each record-in the previous record-out; a zero-length clip is floored to one frame, because an event whose record-in equals its record-out is one no conform will accept.

Exporting from the Timeline sends only clips that are *approved and have video*; if none qualify, it falls back to every clip. The stitcher fetches each approved clip, hands the blobs to `SBFFmpeg.stitchBlobs`, and packages a ZIP through JSZip if that fails.

> **Formats this module does not produce.** There is no OTIO, no FCPXML or Premiere XML, and no CSV anywhere in `timeline/timeline-export.js`. If your finishing partner needs one, the EDL and the project JSON are what you convert from.

Why your labels come back without their line breaks

An EDL is line-oriented, and every field is interpolated into it by concatenation. A newline inside a clip label therefore does not produce a long label — it ends the comment and starts a record. A project name of "Heist" followed by a newline and `FCM: DROP FRAME` emits that line above the real one, and CMX readers take the first, reinterpreting the whole reel at the wrong timecode base; a label containing a plausible event line forges a cut that was never made. Every interpolated field therefore passes through `edlField` in `js/safe-url.js` on its way into the file: carriage returns, newlines and the Unicode line separators U+2028 and U+2029 are folded to single spaces, control characters removed, the result trimmed and truncated at 200 characters. The reel identifier is separately synthesised as `CLIP_nn` from the clip number, so it is never your text at all.

> **This is deliberate, not corruption.** A clip label that appears in an exported EDL with its line breaks flattened to spaces, or cut off at 200 characters, is the export doing exactly what it was built to do. Your labels in the project are untouched — only the copy inside the EDL is folded, and the full text still lives in the project JSON.

The frame rate

Every conversion passes through `normFps`, so an absent, zero, negative or nonsensical rate becomes 24 in one place rather than differently at each call site. An unstated rate is therefore not treated as unknown — it is stamped as 24 with total confidence, and a 30 fps master turned over at 24 lands 25 per cent long with no error anywhere.

> **The Master frame rate, and what it stamps.** The Export panel's picker (`tlFps`, 23.976 through 30) is read at the moment of export and passed to both the EDL and the project JSON. It was not always: for a period nothing read the element at all, the state's `fps` was set to 24 and never read again, and both export calls passed no rate — so every file was stamped 24 whatever the picker showed, and a 25 or 29.97 master conformed a frame adrift per second with no hint in the file. That was found while this chapter was being written and is fixed; the wiring is now pinned by `scripts/test_timeline_export.mjs`.
>
> Note that the stamped rate is an *integer*: 29.97 is written as 30 and 23.976 as 24, because non-drop timecode counts whole frames and the `FCM` line carries the drop/non-drop distinction. That is convention, not rounding error. Read the `* FRAME RATE:` line before you send an export to a conform regardless — an EDL carries no rate of its own, so a conform at the wrong one is silent.

Two Node suites assert this in isolation. `scripts/test_timeline_export.mjs` checks the timecode arithmetic at real cut lengths — an hour, ninety minutes, sub-second trims — because under a minute a wrong hours field looks like a right one. `scripts/test_budget_engines.mjs` enforces that one file defines `SBBudget` and that every page loading it also loads `timeline-doc.js`, so a documentary cannot be priced differently on two screens.

Boards — Storyboards, Shot Lists & Key Art

The breakdown says what the film contains; the schedule says when it is shot. Boards is the room between them, where a scene becomes a set of frames a crew can be handed: a shot list with sizes and lenses, a board with pictures in it, a timed animatic, and the one-sheet it is eventually sold on.

> **How this chapter was written.** By reading `boards/lib-shots.js`, `boards/boards.js`, `boards/boards.css`, `boards/index.html`, `js/lib-scenes.js` and the assertions in `scripts/test_modules.mjs` — not by operating the software. The platform is under active development. Where this manual and the running application disagree, believe the application.

The page is `boards/index.html`, and it carries two tabs that share nothing but the page: **Storyboards** and **Key art**. The engine behind both is `boards/lib-shots.js`, which publishes `CShots` and touches neither the DOM nor the network — the part the node suite exercises directly. `boards/boards.js` is the interface around it, plus the frame sources, the animatic encoder and the poster compositor.

Scenes on the rail

Each row on the left rail shows a scene's slug, its shot count, and how many of those shots have a frame image; beneath the list sit the totals — scenes, shots, and the approximate animatic length in seconds.

⚡ **Seed from script** builds that list from what the platform already knows. `CShots.seedScenes()` looks first at `SB_Timeline_v1` and maps every clip to one board scene, slugged `SC` + the clip number padded to two digits + an em dash + the clip label: `SC01 — Opening`. With no clips it falls back to `SB_Writer_v1` and maps each treatment beat, taking its slugline and the first 200 characters of its body. With neither store populated nothing is seeded. **+ Scene** adds an empty `SCENE n` by hand.

> **Seeding replaces the scene list.** Shots are carried across only where the *slug string matches exactly* — the code indexes the old scenes by slug and reattaches what it finds. Rename a clip in the Studio, renumber it, or re-parse a script so the clips return in a different order, and the matching slug no longer exists: the new scene arrives empty and the shots attached to the old one are gone. No confirmation, no undo.

What the board knows about a scene, and what it does not

`js/lib-scenes.js` is the platform's one scene record: the printed `number` verbatim (`4A`), a numeric `n`, a stable `key`, a `sortKey` that collates 4 < 4A < 5, the cleaned `slug` and the page measure in eighths. Boards does not use it. It calls no `CScenes` function and never reads `SB_Scenes_v1`. Its scene is four fields — `id`, `slug`, `desc`, `shots` — and its identity is the slug string.

Two consequences. The `SC01` on a board is the Studio clip's running number, assigned in `timeline/parser.js` as a counter across every parsed shot; it is not the screenplay's printed scene number, so a revised script's `4A` will not reach the board on its own. And the seeded description is read from a clip field named `prompt`, which the clip record the Studio parser writes does not carry — scenes seeded from the Studio arrive with an empty description.

The shot record

Each shot is one card. A new shot from **+ Shot** starts as a wide, eye-level, static, 35 mm, two-second shot with no frame and no description.

FIELD	STORED AS	VALUES
Size	`size`	EWS · WS · MS · MCU · CU · ECU · OTS · POV · INSERT
Angle	`angle`	Eye level · High · Low · Dutch · Overhead · Ground
Movement	`move`	Static · Pan · Tilt · Push in · Pull out · Track · Handheld · Crane · Zoom
Lens	`lensMm`	millimetres; falls back to 35 if cleared
Duration	`dur`	seconds, half-second steps, minimum 0.5; falls back to 2
Description	`desc`	free text — what happens in the shot
Frame	`img`	an image, held inline as a data URL

The lens is recorded, never enforced: nothing here derives a field of view or warns that an 85 mm insert will not fit the room. It is a note to the camera department, and it travels to the CSV. The card footer moves a shot earlier or later with ◀ and ▶ and deletes it with 🗑.

> **Deletions are immediate.** 🗑 removes a shot and ✕ clears a frame, both without a prompt and both written to storage at once. The only recovery is a project archive taken beforehand.

Suggested coverage

✨ **Suggest coverage** appends a conventional set to the selected scene: a WS master described *Master — full scene*, a 50 mm CU single per named character, then an 85 mm *Insert — key prop / detail*. The node suite pins that shape — two characters produce four shots, the first a master and the last sized INSERT. Two limits before you press it. The cast comes from `characters` in `SB_Timeline_v1` — the whole film's map — and it takes the first four names in it, not the cast of the scene in front of you, so on an ensemble it will propose singles on people who are not in the room. And the set is appended, not reconciled: press it twice and it is there twice.

Getting a picture into the frame

The frame well offers three sources on hover. 📷 **Clip** lists every Studio clip with a rendered video and scrubs it in tenth-of-a-second steps; the captured frame is drawn to a 480×270 canvas and stored as a JPEG data URL at quality 0.65. A clip served cross-origin taints that canvas and the capture fails with a toast saying so. ⬆ takes any image file and centre-crops it to the same 480×270 JPEG. ✨ **AI** posts to your own machine: it needs a bridge URL in `SB_LocalGPU_v1` (Studio → Settings → Local GPU), and sends a prompt built from the scene slug, the shot size, the angle and the description, capped at 400 characters, to `/generate-image`. What that endpoint returns cannot be established from this repository; the board documents only what it accepts — a data URL, or a link it fetches and converts.

Frames live in the board record itself, not in IndexedDB. That is why they are downscaled so hard, and why a failed write raises *Storage full — export a project backup and clear old frames*: read that toast as a warning that the edit you just made was not saved.

The shot list as a file

⬇ **Shot list CSV** writes `shot-list.csv`: one header row — Scene, Shot, Size, Angle, Move, Lens (mm), Secs, Description — and one row per shot, numbered within its scene. Every cell is quoted and internal quotes doubled. A cell opening with `=`, `+`, `-`, `@`, a tab or a carriage return is prefixed with an apostrophe, because a spreadsheet reads those as formulas rather than text; the apostrophe is stripped on display. A description beginning with a dash is the ordinary case that guard exists for.

The animatic

⬇ **Animatic** flattens the whole board — every scene, in order — into timed frames at 1280×720 and 12 frames per second. Each frame holds for its shot's duration, floored at half a second, with the scene slug and shot number burned into the lower left. Unframed shots are not skipped: the interface asks for empty frames included, and each renders as a navy slate carrying the scene label and the first 80 characters of the description, which is what makes an animatic usable before anything has been drawn.

Where the browser supports WebCodecs the frames are encoded as H.264 at 2.5 Mbit/s with a keyframe every two seconds and muxed to MP4 by `editor/lib-mp4.js`. Where it does not, the canvas is recorded in real time to WebM — a three-minute animatic takes three minutes. The file is named from the Studio's project name; with no Studio project stored, that resolves to `animatic-undefined`.

Key art

The second tab composites a one-sheet at 2:3 — 800×1200 on screen, exported at double scale to a 1600×2400 PNG named from the title, or `poster-onesheet.png` when there is none. The form takes a title, a tagline and a billing block of one line per row. *Title low* sets the title at 78% of the height, *Title high* at 16%, with the gradient scrim flipped to match; the background comes from an uploaded image or from the same frame-grab dialogue used on the boards. The title is uppercased in Cinzel, the tagline and billing block in Inter, over the navy ground — webfonts the page loads, so a poster exported without them falls back to generic serif and sans.

Printing a board

🖨 **Print boards** calls the browser's own print. What reaches paper is governed by a real `@media print` block at the foot of `boards/boards.css`, and it is worth knowing what that block does before sending sixty pages to a device.

- The background becomes white, and everything that is chrome rather than content is removed: the top bar, the scene rail, the toolbar, the toast and the frame-grab dialogue.
- The layout stops being a scrolling pane and becomes a block that flows across pages.
- The grid is fixed at **three cards per row**, and a card is told not to break across a page boundary — a shot is never split between two sheets.

- Cards take a plain grey rule, and an empty frame prints as a light grey well rather than the black rectangle it is on screen.
- The selects, the lens and duration inputs and the description print as black text on white with a light border: values, not dark interface widgets.

The white paper is deliberate. The brand specification forbids pure black and pure white as dominant screen surfaces and exempts `@media print` for exactly this case — the manual you are holding is built on the same exemption. Ink on white is the right medium for a page that goes into a folder on a set.

> **Two limits of the printed board.** The grid holds the *selected scene only*, so a print produces that scene's board and nothing else — a sixty-scene film is sixty print operations. And the stylesheet reserves a header element, `.bd-print-head`, that no page in the module creates: the sheet carries no production title, no scene slug and no date beyond whatever the browser's own print header supplies. Write the scene on the page before it leaves the printer.

What is stored, and where

KEY	HOLDS	WRITTEN BY
`SB_Boards_v1`	the scene list, and every shot with its size, angle, move, lens, duration, description and frame image	Boards
`SB_KeyArt_v1`	poster title, tagline, billing block, layout and background image	Boards
`SB_Timeline_v1`	read only — clips to seed from, the cast map for coverage, rendered clips for frame grabs, the project name for the animatic filename	the Studio
`SB_Writer_v1`	read only — treatment beats, used when the Studio holds no clips	the Writer
`SB_LocalGPU_v1`	read only — the bridge address and its key, for ✨ AI	Studio settings

All of it is browser storage on the machine in front of you. Boards and key art travel with the production: both match the portable `SB_*_v#` pattern the vault snapshots, so they ride in a project archive and swap when you switch productions. `SB_LocalGPU_v1` is deliberately excluded — it holds a credential and belongs to the machine, not to the film. An archive restored from another owner has hostile image values blanked and identifier fields stripped of the characters that break out of an HTML attribute, which is why a frame from an untrusted archive may come back empty rather than merely broken.

The Producer Suite I — The Budget Top Sheet

A top sheet is a promise about money made in public. This chapter describes exactly how Cinamate arrives at every figure on it — which numbers you type, which the sheet derives, and the order the derivations run in, because that order moves the grand total.

> **On the accuracy of this chapter.** Everything below was established by reading the source files named in it. The platform is being worked on continuously; if a figure on your screen contradicts a sentence here, the running software is the authority and this page is out of date.

Three levels, one stored sheet

The Budget tab of the Producer Suite (`producer/index.html`, driven by `producer/budget-sheet.js`) is a three-level hierarchy: the *top sheet* on the left, a list of account categories; the *detail table* on the right, the line items inside whichever category is selected; and beneath both, the derived totals. Selecting a row on the left changes what the right-hand table edits.

There is exactly one sheet. It autosaves to browser storage under the key `SB_BudgetSheet_v1`, on a 300 ms debounce after any edit. There is no version history and no undo — a replaced sheet is gone unless you exported it first.

The chart of accounts

Account numbers here are not local to this page. They come from `js/lib-money-accounts.js`, the single chart every module that posts money shares: crew deal memos, the casting office, VFX bids and the Money Room cost report all commit against these same numbers, which is what lets an actual find its way back to the budget line that anticipated it.

A new sheet carries nineteen major accounts. Eleven are *labour* accounts, and that flag defines the base payroll fringes are computed on.

ACCT	CATEGORY	LABOUR	ACCT	CATEGORY	LABOUR
1000	Story & Rights		11000	Makeup & Hair	yes
2000	Producers Unit	yes	12000	Transportation	yes
3000	Direction	yes	13000	Locations	
4000	Cast	yes	14000	Media & Stock	
5000	Production Staff	yes	15000	Post-Production	
6000	Camera	yes	16000	Insurance & Legal	
7000	Sound	yes	17000	Publicity	
8000	Grip & Electric	yes	18000	General Expenses	
9000	Art Department	yes	20000	Payroll Fringes	
10000	Wardrobe	yes	19000	Contingency	computed

Two absences are deliberate. **19000 Contingency** is not a category you can type into — it is derived from everything else, so seeding explicitly discards any contingency line offered to it. **20000 Payroll Fringes** exists as its own account precisely because fringes are cross-departmental; folded into General Expenses they read as several times the whole of an account whose job is to be a small percentage of below-the-line.

The chart also carries *detail* accounts that departments post to, each rolling up to exactly one major above: 4100/4200/4400/4500 under Cast, 8500 under Grip & Electric, 9100 and 9900 under Art Department, 13500 under Locations, 15200/15400/15600/15800 under Post-Production, 16500/16800 under Insurance & Legal. Anything the chart does not recognise rolls up to itself, so it surfaces on the cost report as `Unbudgeted` rather than disappearing into a real line.

Line items — Amt × Units × Rate

A line item stores a description, three calculator fields (Amt, Units, Rate), an Estimated figure, an Actual figure and free-text notes. The rule deciding what a line is worth lives once, in `js/lib-money-sheet.js`:

- If **all three** of Amt, Units and Rate are greater than zero, the line is worth Amt × Units × Rate, and the Estimated cell becomes a read-only computed display.
- Otherwise the line is worth whatever is typed into Estimated, which stays an editable field. This is the lump-sum path.

A zero or a negative in any of the three calculator fields therefore switches the line back to the manual figure. Numbers are read tolerantly — `1234.56`, `$1,234.56` and `1 234.56` all parse the same, and accounting parentheses `(50)` read as minus fifty. Anything genuinely unreadable reads as zero; a budget cell is never allowed to become `NaN`.

On save, the computed figure is written back onto the line item's stored Estimated value. That is not cosmetic: five readers across finance, the investor letter and the learning layer sum the stored value alone, so before this a fully built sheet of calculator lines read as zero everywhere outside this page.

Category subtotals and the percentage column

A category subtotal is the sum of its line items' values, with an Actual subtotal alongside it summed the same way. The percentage at the right of each top-sheet row is that category's share of the *grand total* — not of the direct subtotal — rounded to a whole percent.

The derived layers, and the order they run in

Four figures are computed rather than typed. The order matters, and it is fixed in `sheetTotals()`:

1. **Direct subtotal** — every line item in every category.
2. **Payroll fringes** — the fringes percentage applied to the *labour base*, which is the subtotal of the eleven labour accounts only. Not the whole sheet.
3. **Completion bond** — the bond percentage applied to the direct subtotal.
4. **Insurance** — the insurance percentage applied to the direct subtotal.
5. **Contingency** — applied last, to the sum of the four figures above. Contingency therefore covers the fringes, the bond and the insurance, not merely the direct costs.

Bond and insurance quote on the direct subtotal and nothing else, so raising the fringes percentage does not move either of them. Worked through on a round million, with a labour base of $400,000:

LAYER	BASE	RATE	AMOUNT
Direct subtotal	—	—	1,000,000.00
labour base (a subset, not added again)	—	—	*400,000.00*
Payroll fringes	labour base	28%	112,000.00
Completion bond	direct subtotal	2%	20,000.00
Insurance	direct subtotal	2.5%	25,000.00
Contingency basis	sum of the four	—	1,157,000.00
Contingency	basis	10%	115,700.00
Grand total			**1,272,700.00**

Had contingency been taken on the direct subtotal instead — the order most hand-built spreadsheets use — it would be $100,000 and the grand total $1,257,000. The same inputs, $15,700 apart. Before arguing about someone else's top sheet, establish which order produced it.

All four percentages are edited in the header strip above the top sheet, each clamped: contingency 0–30%, fringes 0–45%, bond 0–6%, insurance 0–8%. A value outside the range is pulled back to the nearest end rather than rejected.

> **Fringes default to zero, on purpose.** A new sheet's fringes, bond and insurance percentages are all zero, while the default skeleton already contains a *Cast fringes* line under 4000 and a *Payroll fringes (labor accounts)* line under 20000. Fill in those line items or set the percentage — doing both counts your fringes twice, and nothing warns you.

Why every figure is held in cents

All arithmetic on this sheet runs through `js/lib-money-math.js`, which carries money as an integer number of cents. Rounding happens once, half away from zero, at the moment a fraction of a cent is created — a multiplication or a percentage — and never on a sum. A total is then the sum of the cents that were actually posted, so it foots by construction rather than by luck.

This is not fastidiousness. A JavaScript number is a binary float: 3 × 5 × 1061.64 is 15,924.6, and left as a float it prints as `15924.599999999999`. Rounding each row before adding, rather than adding what was posted, produced a cost report whose TOTAL row disagreed with its own columns by tens of dollars across a couple of hundred accounts — on the report that goes to the completion bond.

Money therefore leaves the platform as a fixed two-decimal *string*, not as a raw number, and that is what makes an exported column safe to sum. `scripts/test_ops_money.mjs` asserts it: every money cell in an exported report must match `-?\d+\.\d{2}`, with no cell anywhere carrying three or more decimals.

Actuals against estimates

Every line item carries an Actual beside its estimate, and category and sheet actuals are summed from those. When the sheet holds any actual, the grand total row prints it alongside the estimate.

If the Money Room has posted anything — purchase orders, petty cash or payroll in `SB_Money_v1` — the top sheet also prints a one-line cost summary beneath the totals: estimated final cost, and the variance marked *over* or *under*. It is computed by the Money Room's own cost report, not recomputed here.

Actuals also feed the estimator's learning layer (`js/learn.js`, stored under `CIN_Learn_v1`), which learns a per-account correction from lines carrying both an estimate and a real actual. It is gated: a production teaches once, when marked wrapped in `CIN_Projects_v1`. Mid-shoot, a department that has spent 12% of its budget in week two looks like an 88% underrun, and folding that in would drag every future estimate down.

Cash needed by phase

Below the grand total the sheet prints a three-way split — prep, shoot, post — of when the money leaves the bank. Each account carries a fixed phase profile (rights entirely prep, post-production almost entirely post, cast overwhelmingly shoot); the derived layers are split between shoot and post, the last bucket absorbing the remainder so the three figures add back to the grand total exactly.

The norms advisor

Each top-sheet row is measured against a band of typical shares of the grand total for that account, held in the `NORMS` table in `producer/budget-sheet.js`. A category outside its band gets a marker — ▲ heavy, ▽ light — with the band in the marker's tooltip, and a line beneath the totals counts how many accounts are flagged. Being outside a band is not an error. It is the question the bond company will ask, asked early.

Seeding from the script estimate

Seed from script estimate runs the script-driven estimator over the current project, routes every estimated line to the top-sheet category its account rolls up to, and writes the **midpoint** of each low–high range into the line, preserving the range in that line's notes. Where the learning layer holds at least two past actuals on an account and a correction other than 1.0, the seeded figure is multiplied by it and the note records by how much. Contingency lines are dropped, since contingency is derived. Where the Schedule tab's stripboard carries real day assignments the estimator uses them, and the confirmation reports what the cast drops saved in weeks.

> **Seeding replaces the whole sheet.** Every line item in every category is deleted before the estimated lines are written, and the contingency percentage is reset to 10%. Anything you had typed by hand is lost, including actuals. Export first — *Save (.json)* — if the sheet holds work you cannot retype.

The CSV export

Export CSV downloads the sheet as `<sheet name>.csv`, the name stripped of anything outside letters, digits, hyphen and space. Columns: `Account`, `Category`, `Line item`, `Amt`, `Units`, `Rate`, `Estimated`, `Actual`, `Notes`. Each category writes one row per line item followed by a `SUBTOTAL` row when it holds anything. At the foot: `SUBTOTAL (all categories)`; a fringes, bond and insurance row each, but only when that percentage produced a figure; a contingency row always; and `GRAND TOTAL`. The Actual column is left empty rather than written as `0.00` when there is nothing in it.

Estimated and Actual are written as fixed two-decimal strings, for the reason given above; a spreadsheet reads them as numbers and sums them correctly.

The leading apostrophe is deliberate

A cell whose first character is `=`, `+`, `-`, `@`, a tab or a carriage return is treated by Excel and Google Sheets as a *formula*, not as text. A line item description typed into this sheet would then execute on the machine of whoever opened the export. The exporter therefore prefixes such a cell with a single apostrophe, which both spreadsheets read as "the rest of this cell is text".

> **Do not strip those apostrophes.** If you see a cell reading `'=Second unit day`, that is the neutralisation working as designed. One consequence to expect: a genuinely negative money figure exports as `'-1234.56`, because it too begins with a hyphen, and a spreadsheet will treat it as text rather than as a number. If you need a credit to behave arithmetically in Excel, re-enter that one cell after opening the file.

Saving, loading, printing, starting over

Save (.json) writes the complete sheet — categories, line items, percentages, actuals — to `<sheet name>.budget.json`. That file is the archive copy; the CSV is a report, and cannot be loaded back. *Print / PDF* hands the rendered sheet to the browser's print dialogue.

> **Load and New sheet both replace the live sheet.** *Load* accepts a `.json` file and, if it parses and contains a categories array, swaps it in immediately — with no confirmation prompt. *New sheet* asks first ("Start a new blank top sheet? The current sheet will be replaced (save it first if needed).") and then discards everything. Both write straight to `SB_BudgetSheet_v1`. There is no undo.

What this sheet is not

Stated plainly, because a document full of account numbers invites the assumption:

- **It is not an accounting system.** One estimate and one actual per line, with no ledger, no double entry, no reconciliation and no audit trail. The Money Room tracks commitments and spend; neither is a set of books.
- **It files nothing.** Nothing here is submitted to a jurisdiction, a payroll company, a guild, a bond company or a tax authority. Every export is a file that lands in your downloads folder.
- **The incentive figures are estimates from a table.** The Incentives tab computes recovery as budget × qualified-spend percentage × credit rate against a stored table of headline program terms, and flags a budget below a program's minimum spend or above its cap. Real programs carry qualification rules, allocation queues, sunset dates and audits that no table models. Treat the output as a shortlist for a production accountant, never as advice.
- **The on-screen figures are abbreviated.** The top sheet shortens money — a grand total of $1,272,700 reads as `$1.3M`, a line of $15,924.60 as `$15.9k` — and clamps a negative to zero for display. The exact figures live in the CSV and the JSON. Quote from the export, not from the screen.

The Producer Suite II — Scheduling, DOOD & Call Sheets

A broken-down script is a list of scenes. A schedule is a claim about how many days those scenes will take, who is owed money on which of them, and what time a hundred people arrive tomorrow. The Schedule tab makes all three from the same board — and it will tell you, in as many words, when the first of them is still a guess.

> **How this chapter was written.** By reading the platform's source, not by driving it. Cinamate is under active development. Where a screen and a paragraph here disagree, believe the screen.

The stripboard

The board lives in the Schedule tab of the Producer Suite and is stored under one key, `SB_ScheduleBoard_v1`. Everything below writes there.

Seed scenes from script builds the strips. Each slugline in the timeline's script becomes one strip, sized in eighths of a page at five script lines to the eighth (`LINES_PER_EIGHTH` in `timeline/timeline-budget.js`), with a one-eighth floor. A strip is marked *night* when its slugline contains NIGHT, DUSK, EVENING, MIDNIGHT or PRE-DAWN, and *day* otherwise. Cast comes from the estimator's per-scene map — a ranked character name occurring in that scene's text. New strips land on day `-1`: the boneyard.

> **Seeding replaces the board.** Seed scenes from script discards the existing strips outright and asks nothing first. Breakdown tags, extras counts, scene notes, hand-edited page counts and every day assignment go with them. Re-seed when the script has genuinely changed, not to refresh the view.

Double-clicking a strip opens its breakdown: slugline, page count, day or night, background count, cast, notes, and six tags — stunts, SFX, VFX, water, animals, vehicles. The page field accepts what an AD actually writes: `1 7/8`, `7/8`, `15`, or a decimal like `2.5`. Deleting a scene from that panel asks first.

Strips are ordered two ways. Drag them between the boneyard and any day — there is always one empty trailing day to drop into. Or press **Auto-schedule**, which fills days at the target pace in *script order*, or in *group by location* order, which sorts by the set name parsed out of the slugline and puts day work before night work at each location: fewer company moves, the way a board is really laid out. Scene numbers are preserved either way.

> **Unschedule all** returns every strip on the board to the boneyard immediately, with no confirmation and no undo. The strips and their breakdowns survive; the day assignments do not.

Shoot days are keyed on date *and* unit

A day on the board is an index. A day on set is a date. The record that joins them is `SB_ShootDays_v1`, in `js/lib-shootdays.js`, and it holds exactly this: `dayIdx`, `date` as `YYYY-MM-DD`, `unit`, the `sceneIds` scheduled that day, `wrapped`, and `pinned`.

The identity is the pair — date and unit, never the date alone. Main unit and a splinter unit can both shoot one date; that is two days' work, two call sheets and two rows on the daily report. Lookups therefore take a unit alongside the date, and `unitsOnDate` lists every unit working a given date. Asked without a unit, the lookup answers with the first record on that date, which is the main unit because indices sort ascending. The default unit is `MAIN`.

Dates come from the **Day planner**, the bar `tools/sched-weather.js` injects above the board. It takes day 1's date, a city or a latitude and longitude, a skip-weekends switch and an optional day count, and saves them to `SB_ShootPlan_v1`. From there, day *n* is *n* shoot days forward of day 1, stepping over Saturdays and Sundays unless skip-weekends is off, and computed at noon UTC so a browser in any timezone lands on the same calendar day.

Pinning is the exception to that arithmetic. A record is pinned when a writer *stated* its date and unit rather than deriving them — which is what happens when Dailies names the day it is shooting. A rebuild recomputes the derived halves of every record and leaves the pinned halves alone, so a take dated to a day the stripboard has never heard of still resolves.

The pace, and why it stays a guess until you wrap a day

The board ships with a planning pace of **4.5 pages per day**. It is a default for a film nobody has shot yet, and it is deliberately the only place that number appears.

The board can do better than that: it can learn the pace this production actually shoots. But it learns from one thing only — **shoot days an operator has explicitly marked wrapped**. Until three such days exist, the board reports the shipped default and says so in plain words on the chip beside the toolbar:

```
target 4.5 pg/day — the shipped default; nothing learned yet (no wrapped days)
target 4.5 pg/day — the shipped default; 2 wrapped days so far, 3 needed to learn
target 3.25 pg/day — learned from 4 wrapped days
```

An operator who never wraps a day will read the first of those lines for the whole shoot, and every day count on the board — and every number downstream of it — will rest on a figure nobody measured.

How to wrap a day

The control is on **Today**, the on-set page, not on the board. Beside the day selector is a button reading ◉ **Wrap day**. Pick the day, press it, and confirm the prompt; the button then reads ↺ **Re-open day** and reverses the same flag. That is the only shipped control that writes `wrapped`, and it is deliberately on the page the people standing on the day are looking at: whether a day with no takes logged against it is an unshot day or a wrapped one is something only they know.

The board shows the result but does not set it. A wrapped day carries a small read-only WRAPPED chip in its header on the stripboard, and the same word appears on that day's call sheet. There is no toggle there.

What is learned, exactly

For every wrapped day, in index order, the board compares two numbers: the eighths *planned* on it from the strips, and the eighths it *achieved*, read from the take log. Takes are gathered from both stores that hold them — `SB_TakeLog_v1` and `SB_Dailies_v1` — and matched to the day by its calendar date. Each take names a scene; that scene's eighths are counted **once for the production**, however many takes it took and however many days it was touched on. A day that re-shoots a scene already in the can reports it as a pick-up worth zero new pages, because the pace being measured answers "how many days to cover the script".

The learned pace is the **median** of the achieved column, divided by eight, clamped between 1 and 12 pages. A median, not a mean, because a single catastrophic day should not move the target on its own — and three is the smallest count at which the median is a real order statistic. A wrapped day whose take log is empty is not evidence at all: reading that as zero pages would teach the board that a production which does not log takes shoots nothing.

A target you type always wins. Setting the Target pages/day field marks the pace as yours and the board schedules at it, while still reporting what your wrapped days achieved next to it. **Use learned pace** gives the override back.

> **One caveat on un-wrapping, established by reading.** Re-opening a day reverses the flag in `SB_ShootDays_v1`, and the pace model recomputed from scratch drops that day's evidence — this is proved by the suites. The board, however, keeps its own merged pace log inside `SB_ScheduleBoard_v1`, and the merge can only retract a row when the caller tells it which day indices it examined. The board's single call site (`producer/schedule-board.js`, in `syncLearning`) does not pass that argument, so a row written while a day was wrapped appears to survive the un-wrap in the stored log. Treat wrapping as a statement of fact rather than a switch to try.

Day-out-of-Days

The DOOD button opens a grid: one row per cast member, one column per shoot day, sorted by days worked. The letters **compose** rather than branch, so a cell can say everything true of that day at once:

CODE	MEANING
S	Start — the performer's first day
P	Pick-up — re-engaged after a drop
W	Work
D	Drop — released after this day
F	Finish — the last day of the engagement
H	Hold — an idle day inside an engagement, still billed
—	Dropped — an idle day the production released, not billed

So SW, WF and SWF read as they always did, and SWD ("starts, works and is dropped") or PWF ("picked up, works, finishes") can be said at all. The trailing columns are TOT span, WRK worked, HLD billed holds, DRP dropped days and WKS weeks billed, with the weeks a drop saved shown against the row.

Which idle stretches qualify as a drop is not this board's decision. It calls `SBBudget.castWeeks` in `timeline/timeline-budget.js`, the single implementation of that arithmetic on the platform, so the letters on

the grid and the money on the budget cannot disagree. Its shipped terms: ten free days minimum before a gap is droppable, one drop per performer, one day's written notice, a one-week minimum per segment. Notice falling before day 1 is served at engagement and the drop stands; notice falling on a day already shot is refused, and those idle days stay billed as holds rather than quoting a saving nobody can legally take.

The **working week** selector accepts 5, 6 or 7 days. It changes the calendar — the same days in fewer weeks — and the money: the sixth- and seventh-day premiums are read from the timecard engine's defaults, so a six-day week averages (5 + 1.5) ÷ 6 per crew day. If the money library is not loaded on the page, the premium is reported as unavailable rather than quietly applied as 1×.

Where day counts reach the estimate

When the Budget tab seeds from the script estimate, it asks the board for overrides first. A scheduled board hands back exact cast spans per performer, the working week, and special-unit day counts read off the breakdown tags — stunt, pyro, water and animal days, plus a background total — which override the estimator's keyword heuristics and its script-order guess at the DOOD. An unscheduled board hands back nothing, and the estimator falls back to its own model: page count over a scale pace, multiplied by a difficulty load, with a five-day floor. That fallback pace is the estimator's own, not the board's learned one.

The call sheet

Every day header carries a clipboard button, and it opens that day's sheet. Almost nothing on it is typed: the sheet is assembled from fields that already existed one hop away.

BLOCK	COMES FROM
Date, unit, wrapped	`SB_ShootDays_v1`
Scenes, pages, cast, tags	the strips on that day
Sunrise, sunset, golden hours	computed locally from the pin on `SB_ShootPlan_v1`, at the location's civil offset
Sets, addresses, parking, load-in, hospital	`SB_ScoutBook_v1`
Department calls, walkie channels	`SB_Crew_v1`
Meals, wrap, turnaround	the timecard engine's defaults
Advance schedule	the next two days on the board

The whole clock derives from one number you enter: the general crew call. Shooting call is 30 minutes after it, breakfast 30 before it, lunch six hours from call for 30 minutes, second meal six hours after that, estimated wrap on the twelve-hour convention plus the meal, and the earliest next call ten hours after wrap — the same rules the timecard engine bills against, so the sheet cannot promise a meal that payroll then penalises. Departments offset from the crew call: art and hair-and-make-up an hour early, wardrobe 45 minutes, grip-and-electric and production 30, camera and sound with the unit. Walkie channels go only to departments this show actually crews, production on channel 1.

Individual cast calls come from the board: the eighths scheduled before a performer's first scene that day, pro-rated across the shooting window, backed off an hour for hair, make-up and wardrobe, rounded to five minutes. A

per-name override stored on the day is honoured if present, though no shipped control writes one. Each DOOD letter is spelled out — START, WORK, PICK-UP, HOLD, DROP AFTER TODAY, FINISH — because the sheet goes to everyone, not only to an AD, and a drop day carries the date the notice is due.

Anything the sheet was not handed is listed at the foot under *Not on this sheet, and why*, naming the store to fix: no crew directory, no hospital on file, no location pin, no call time. The forecast is the one line that cannot be computed on the machine; it is fetched live, and a blocked or failed request says so rather than impersonating clear skies.

Revisions are derived, not remembered. The sheet's content is reduced to a signature; re-printing an unchanged sheet is not a revision, while a sheet that has moved since its last issue is labelled REV. A PENDING until it is re-issued. Issue / print stamps the issue and prints; issues are kept on the day and capped at 26.

Getting it off the screen

Print / PDF on the toolbar opens the Day-out-of-Days and prints the board with it; the call sheet has its own Issue / print. There is *no* CSV export on the Schedule tab. The Export CSV button in the Producer Suite belongs to the Budget tab and exports the top sheet, not the schedule — print to PDF is the schedule's export path.

What this scheduler does not do

- It does not know about cast availability it was not told about. Nothing on the board says a performer is on another film in week three.
- It does not book, hire, hold or notify anyone. A drop notice date is arithmetic; serving the notice is a person's job.
- It does not send a call sheet. It draws one and prints it.
- It does not enforce a turnaround, a meal or a sixth day — it flags a short turnaround and quotes the rule it measured against.
- It does not know the weather. It quotes a forecast, or says it could not get one.

A generated call sheet is a **draft for a human first AD to check**. It is assembled correctly from what the platform holds, which is a different claim from being right about tomorrow.

What is guaranteed

Two suites pin this chapter, both running offline against the libraries. `scripts/test_shootdays.mjs` holds 95 assertions on the shoot-day record: the weekend-skipping calendar, the date-and-unit key, two units sharing one date, pinning surviving a rebuild, wrapping and un-wrapping through the store. `scripts/test_schedule_learn.mjs` holds 145 on the schedule: that three takes of one scene count its pages once, that only wrapped days become evidence, that the learned pace is the median and not the mean, that a typed pace overrules a learned one, and that every block of the call sheet is what it claims to be.

Casting & The Cast Desk

Casting is the first department that sends your screenplay to people outside the production. Everything in this chapter — the role list, the pipeline, the sides, the lookups — is downstream of that one fact, and the chapter is written so that you know exactly what leaves the building.

> **On the accuracy of this chapter.** It was written by reading Cinamate's source rather than by operating the software. The platform changes; when a screen contradicts a paragraph here, the screen is the authority.

Two rooms, three stores

Casting is not one page. The **Casting Office** at `casting/index.html` is the cast desk proper — roles, candidates, sides, offer memos — driven by `casting/lib-castdesk.js` (`CCastDesk`) and stored under one key, `SB_CastingDesk_v1`. The **Production Office** at `production/index.html` carries a second Casting & Sides pane with its own registers, `SB_Roles_v1` and `SB_Candidates_v1`, its own sides cutter in `production/lib-prod.js`, and the Cast Intelligence lookups in `production/lib-cast.js`.

These stores do not synchronise. A candidate entered on the cast desk is invisible to the production office register, and Cast Intelligence's **Add as candidate** writes to `SB_Candidates_v1`, not to the cast desk. Pick one as your book of record and keep it.

Roles from the screenplay

Seed roles from script reads `scriptText` out of `SB_Timeline_v1` — the working screenplay from the Studio — and needs at least 40 non-whitespace characters before it will attempt anything. `CCastDesk.charactersFromScript` then splits the script through the one scene model in `js/lib-scenes.js` and counts dialogue cues.

A cue is decided by `CScenes.cueName`, which every module now shares. The line must be ALL-CAPS, between 2 and 30 characters after any `(V.O.)`, `(O.S.)`, `(O.C.)`, `(CONT'D)`, VOICE OVER, OFF SCREEN, PRE-LAP or FILTERED suffix is stripped; it must not parse as a slugline; it must not end in a colon, semicolon, exclamation or question mark; it must not open with a format keyword (FADE IN, CUT TO, DISSOLVE TO, INTERCUT, MONTAGE, TITLE, SUPER, CHYRON, ANGLE ON, CLOSE ON, INSERT, OMITTED, THE END and the rest of `NOT_CUES`); and it must match `^[A-Z][A-Z0-9 .,'\-]*$`. The cast desk adds one more test of its own: the cue must be followed by a non-blank line that is not itself a slugline, so a lone shouted word with nothing under it is not promoted to a character.

Each role comes back with its name, its cue count, the number of scenes it speaks in and the scene list, sorted by cue count descending and then by name. That ordering is the first useful thing on the page: the names at the top are your leads, and the long tail of one-cue roles is your day player list. The board still needs curating — a short shouted action line with dialogue-shaped text under it will land in the list, and a character who never speaks never appears at all. Prune by hand, and add what the parser missed with **Add role**, which marks the role `manual`.

> **Re-seeding rebuilds the board and can take candidates with it.** On a re-seed, roles marked `manual` survive, and a seeded role whose name still appears in the script keeps its candidate list. Any seeded role the parser no longer finds — because the character was cut, or because you renamed the row in place — is dropped outright, and its candidates, contacts, hold dates and notes go with it. There is no confirmation and no undo. The row-level `×` on a role deletes it and its candidates the same way.

What the desk tracks per role

Each role carries a candidate list, and each candidate carries: name, contact, self-tape link, status, hold-from, hold-to, rate and notes. Nothing here is looked up. Contacts and tape links are whatever you paste; a tape link is rendered as a clickable button only when it begins with `http://` or `https://`, and then only through the platform's URL sanitiser.

The pipeline is a fixed ladder, `CCastDesk.STATUSES`: *submitted* → *callback* → *test* → *offer* → *hold* → *booked* / *released*. Nothing enforces the order; the status is a label you set. A role shows as *booked* once any candidate reaches that status, otherwise as *n in play* when offers or holds exist, otherwise as *open*. The strip across the top is `boardSummary`: a count per status plus the totals — candidates, roles, and roles with a booking.

Hold conflicts

`holdConflicts` compares every candidate on the whole board, not just the selected role. Two entries conflict when the names match case-insensitively, both are at *hold* or *booked*, both carry a from and a to date, and the ranges overlap. Dates are compared as ISO text, and the endpoints are inclusive: a hold ending 10 October and one beginning 10 October is a conflict. A candidate at *submitted* never conflicts, and missing dates are skipped rather than guessed. The warning names both roles and both ranges; resolving it is yours to do.

Audition sides — what is actually in them

`CCastDesk.sidesFor(scriptText, character)` assembles the sides. It matches the character name case-insensitively, walks the scenes, and keeps every scene in which that name appears as a dialogue cue. For each kept scene it emits a rule reading `— SCENE` and the scene's printed label, then the slugline — or `(no slugline)` — then the scene body. If the character never speaks, the result is an empty string and the page says so.

> **Sides are not redacted, and that is correct.** What comes out is the *full text* of every selected scene: the other characters' cues and dialogue, the action lines, the stage business, all of it. A performer cannot play a scene from their own lines alone, so this is the right behaviour — but it means the file you are about to send to an agent contains complete pages of an unreleased screenplay. Read what you generated before it leaves the browser. Trim it if the scene contains material you are not ready to circulate; the generated text says as much in its own header, and asks you to verify the draft is current first.

Sides are generated in the browser from the working screenplay and are not stored — regenerate after every rewrite. **Copy** puts the text on the clipboard; **Download .txt** saves it as `sides-<role>.txt`.

Two modules cut sides, and they select differently

This is deliberate, and an operator who does not know it will conclude that one of them is broken.

	CAST DESK	PRODUCTION OFFICE
File	`casting/lib-castdesk.js`	`production/lib-prod.js`
Test	`speaksIn` — a dialogue cue in the scene	`appearsIn` — a cue *or* the name in an action line
Purpose	Audition sides	On-set sides
Output	One text block	Array of scenes: label, number, slug, text

A performer auditions with the scenes they perform, so the audition cut asks whether the character speaks. An actor who wrestles someone across a kitchen with no dialogue is still called that day and still needs the pages, so the on-set cut asks whether the character is present at all. Both questions are answered by `js/lib-scenes.js`, which owns `cueName`, `speaksIn` and `appearsIn`, so the two modules disagree only where they are meant to.

`appearsIn` matches whole words, with explicit boundaries rather than a word-break class, so a possessive still counts — MAGGIE'S COAT contains MAGGIE — while a character named AL is no longer found in every scene containing CALL, HALL or GENERAL. The substring match this replaced was, by the source's own account, mailing entire screenplays to representatives who had no reason to receive them.

> **The production office pane still heads its output "AUDITION SIDES".** That block is built on `appearsIn`, so it is an on-set cut regardless of the caption. Take the selection from the module you used, not from the title line.

The offer memo

`offerLetter` prints business points only: production, date, performer and representative, role, engagement dates, rate and unit, billing. Anything left blank prints as `TBD` — no figure, date or name is ever invented. Every memo closes the same way: not a binding agreement, conditional on a long-form agreement reviewed by production counsel and on any required union clearances and work permits. The envelope button on a candidate row pre-fills the memo from that candidate's name, rate and hold dates.

> **Commit to Money Room writes a real commitment.** It parses the digits out of the rate field, refuses anything that is not a positive number, and posts an open purchase order into `SB_Money_v1` against the Cast account — `4000`, from `CAccounts.DEFAULT_CAST` in `js/lib-money-accounts.js`. The caption printed beside that button still says account 1400; the code does not use it, and 1400 exists nowhere in the chart. Believe the toast, which names the account it actually used. The amount posted is the rate as entered — one week or one day — not a computed total for the engagement.

Cast intelligence

The Cast Intelligence block sits at the foot of the production office's Casting & Sides pane: type a performer's name, optionally a target director, and press **Analyze**.

> **This sends a real person's name off the device.** The actor and director names you type are transmitted to `query.wikidata.org`, a public SPARQL endpoint, on every lookup — no key, no account, and the query text carries the name. If you have stored a TMDB key, the name is additionally sent to `api.themoviedb.org` with that key in the request. Nothing else on the cast desk leaves the browser; this feature does. The key itself stays local: `SB_TMDB_v1` is excluded from the cloud snapshot in `js/project-badge.js` and from vault exports by the local-only rule in `projects/lib-vault.js`. Replies are cached in `CIN_Learn_v1` for one week, capped at sixty entries, with markup characters and script-bearing URL schemes stripped on the way in.

Wikidata supplies the filmography, the directors a performer has worked with, and — for **Suggest cast for this director** — the collaborators a director keeps returning to, ranked by count. TMDB, when a key is present, adds billing order and a popularity figure the card labels an audience-demand index.

Fit

`CCast.fit` returns a score out of 100 and the reasons behind it. The arithmetic is plain: up to three shared films with the director at 22 points each, cosine overlap of the two genre profiles scaled to 30, 16 points for a credit within the last two years against 4 if not, and one point per film up to 20. A shared film is matched by film id on the TMDB path and by director name on the Wikidata path. It is a sort order for a shortlist, not a judgement about the role.

The quote estimate, and what it is worth

`CCast.quote` places the performer in one of five named bands — Scale / day player, Established supporting, Name, Star, A-list — using three inputs: credits released in the last six years, how many of those were top-two-billed, and the TMDB popularity figure. The band's low and high are fixed constants in the `TIERS` table, identical for everyone placed in that band; the source describes them as trade-reported convention brackets and states that profit participation is not modelled. The floor quoted in the basis lines is published SAG-AFTRA theatrical scale as recorded in `production/lib-cast.js`. Enter a figure in the *known quote* field and it overrides the model entirely.

Read the basis lines, not the number. The estimate is a bracket derived from public career data, calculated without an agency, a script, a start date, or a single conversation — treat it as a bucket for a shortlist, never as a negotiating position. One structural limit is worth knowing: the Wikidata path carries no billing order and no popularity, so with no TMDB key stored, the top-billed count and the demand index are both zero and no performer can be placed above *Established supporting*, however famous they are.

Scheduling implications

The cast desk's cue counts are the earliest read on the shape of the cast. A role speaking in one or two scenes is a day player — a single engagement, a single call, a cost the estimator budgets on account `4200`. A role carrying cues across twenty scenes is a lead, whose availability constrains the whole board and whose idle days between first and last work day are either held and billed or dropped and released.

Cinamate does not carry that judgement through automatically. The estimator in `timeline/timeline-budget.js` makes its own split — the top two ranked characters are leads, the next six at most are supporting, everything beyond that is a day player — and the Day-Out-of-Days matrix in `producer/schedule-board.js` is built from the cast assigned to the stripboard's strips, not from the cast desk. Neither reads `SB_CastingDesk_v1`.

If you have pruned the role list here, prune the strip cast and the roles register too, or the schedule and the budget will keep counting characters you have already cut.

The one piece of scheduling the desk does enforce is the hold conflict, and it does so only across its own board. It cannot see another production's holds or an availability your candidate has not told you about, so confirm dates with the representatives before you move the board.

What the suites check

`scripts/test_castoffice.mjs` exercises the cast desk engine against a fixture screenplay: every cue-detection rule above, the role counts and scene lists, the hold conflict cases including touching endpoints and missing dates, that sides carry the slugline and the dialogue and exclude scenes without the role, that an unknown character yields nothing, and that an empty offer memo prints `TBD` rather than inventing figures. `scripts/test_advisor.mjs` covers Cast Intelligence: the TMDB and Wikidata parsers, the fit scoring, and the quote tiers including the known-quote override. Both run under `node scripts/run_all_tests.mjs`.

Locations, Permits & The Scout Book

A location is the one department where the production is a guest. Somebody else owns the room, somebody else's neighbours must tolerate the truck, and a city office decides whether the street is yours at all. The Scout Book at `/locations/` is where that becomes a record.

> **How this chapter was written.** By reading the files, not by using the screens. Cinamate is still being built; wherever a screen and a sentence below it disagree, the screen is what runs and this page is out of date.

The location bible

The page is `locations/index.html`; its arithmetic and its directories are `locations/lib-scout.js`, which exports `CScout`. Everything typed is written to one localStorage key, `SB_ScoutBook_v1`, on every change — there is no save button. It holds the locations, the hub picked in each directory, the last sun query and the checklist ticks. A record comes from `blankLocation()`; no field is mandatory, and an empty one stays empty rather than being guessed at.

GROUP	FIELDS
Identity	`name`, `address`, `scenes`, `notes`
Practical	`parking`, `power` (free text, as scouted), `loadIn`
Safety	`hospital`, `hospitalAddress`
Paper	`permitStatus` (none / applied / issued), `releaseStatus` (none / sent / signed)
The pin	`lat`, `lon`, `tzOffsetMin`, `tzSource`
Electrical	`supplyAmps`, `supplyVolts`, `supplyPhases`, `fixtures`
Media	`photos` — a list of ids, never the images themselves

The hospital fields are not decoration. The call-sheet builder in `producer/schedule-board.js` reads `SB_ScoutBook_v1` for the hospital, parking and load-in lines of the safety block, and `today/index.html` matches the day's sets loosely against location names to surface it. A blank hospital field prints as a stated gap on the sheet. Note also that the platform holds *two* location registers — this one and a flatter `SB_Locations_v1` table on the Production page — which do not synchronise. The Scout Book is the one carrying the hospital, the pin and the photographs.

Suggesting locations from the screenplay

⟳ **Suggest from script** reads `SB_Timeline_v1` in this browser and runs `scriptLocations()` over its `scriptText`. It takes every slugline — `INT`, `EXT`, `INT/EXT`, `I/E`, numbered or not — strips the prefix and the time-of-day suffix at the first spaced dash, upper-cases the remainder and groups repeats. An unreadable slugline

becomes `(UNNAMED LOCATION)` rather than disappearing, and nothing enters the book until you press **+ Add to book**.

One caution: the scene numbers it reports are *positional*. It counts sluglines from the top of the file and ignores any number printed in the script, so a scene the screenplay calls 12 is listed as 3 if it is the third slugline. On an unnumbered draft the two agree; on a revised numbered script they will not.

Scout photographs

Adding a photo re-encodes it in the browser: the longest edge is reduced to 1280 pixels if it exceeds that, and the result is written as a JPEG data URL at quality 0.85. The bytes go to IndexedDB — database `cinamate_scout`, object store `photos`, keyed `ph<timestamp><random>`. Only the id goes into the location record, so record and image travel separately.

When they have separated, the page says so: a photo id whose bytes are not in this browser becomes a dashed ⚠ **photo missing** tile naming that id, and the strip gains a line telling you to restore from a backup carrying media. It does not quietly drop the image; that behaviour once made a scout book that had lost every photograph look like one that was never photographed.

> **Scout photographs are other people's property, and they travel.** These are interiors of real premises — very often private homes whose owners let a location manager through the door, not a studio. The bytes sit in IndexedDB unencrypted; they are packed into a `.cinamate` export, a self-contained file with no size limit; and `netlify/functions/projects-sync.js` lists `cinamate_scout/photos` among the three stores the studio cloud accepts, so a push puts them where all five owners of that shared cloud can pull them down. Cloud transfer is capped — 900 KB per record, 1 MB per request, 48 requests, so 48 MB per production — but that is a size limit, not an access limit. Treat a scout book as personally and commercially sensitive.

> **Deleting a location deletes its photographs.** The ✕ on a location card confirms, then removes every photo id it owns from IndexedDB along with the record. The per-photo ✕ deletes those bytes immediately with no confirmation at all. Neither is undoable here; the only recovery is a restore, and only from a backup taken with media.

The tech-scout checklist, and power

`locationChecklist()` returns ten fixed items — power, parking, load-in, bathrooms, neighbors, noise, cell signal, nearest hospital, permits and insurance COI — each with a sentence on what to actually record. The ticks live in `SB_ScoutBook_v1` under `checks`, keyed by item id: one set of ten for the whole production, not one per location, so ticking *Noise* does not record that noise was checked anywhere in particular.

Power is the item with arithmetic behind it: `parseSupply()` reads an amperage out of the scouted note (*100A*, *200 amp 3-phase*, *2 x 20A*) and returns nothing at all when the note holds no number, and `powerDemand()` puts the card's fixture order against that supply as amps per leg, returning *ok*, *tight*, *derate*, *over* or *unknown* against the 80% continuous-load ceiling. An unparseable note is an unknown supply, never a safe one.

The permit directory

`CScout.PERMITS` holds six hubs. Each entry carries the office name, when a permit is required, cost, lead time, insurance, police and traffic extras, an `applyUrl` and a `verified` flag. The rule the data obeys — asserted by `scripts/test_locations.mjs` — is that an unverified entry may never carry an application URL; the page renders a web-search link in place of the **Apply** button. All six entries today are marked verified and carry an official URL.

HUB	OFFICE NAMED IN THE FILE
Toronto	City of Toronto Film & Entertainment Industries office
Vancouver	City of Vancouver Film and Special Events Branch
Atlanta	Atlanta Mayor's Office of Film & Entertainment
Los Angeles	FilmLA
New York	NYC Mayor's Office of Media and Entertainment
London	Borough Film Services — the entry states there is no single London office and no single London fee; it varies by borough

Hubs are matched by `matchHub()`, deliberately loose: a small alias table (`la`, `nyc`, `burnaby`, `gta` and four more), then a hit if a hub name appears anywhere in your text, or if four or more characters you typed prefix one. Free text such as *shooting in downtown toronto, ON* resolves correctly — and so does any other place whose name merely contains a hub name, so read the card's heading before trusting the fees under it. **Use Advisor location** maps the tax-incentive jurisdiction in `SB_Budget_v1` through `INCENTIVE_HUB`: Ontario to Toronto, BC to Vancouver, Georgia to Atlanta, California to Los Angeles, New York to New York, either UK incentive to London. Outside those six the page sends you to the local film office.

Log permit fee commits an amount to the Money Room as an open purchase order against account 13000 (Locations), vendor *Film permit office*, written into `SB_Money_v1`. It is a real budget commitment, and it must be reversed in the Money Room rather than here.

> **The directory text is truncated, and a footnote contradicts the card above it.** In every one of the six permit entries the `required` field ends mid-sentence — several end mid-word — and other fields do the same. The page prints them verbatim, so you read a rule that stops before its exception. And the card's footnote states that this build could verify neither fees nor application URLs and that none are shown, while the card above it shows both. Nothing here is a quote or a legal position: confirm the fee, the lead time and the insurance wording on the office's own page.

The stage and facility directory

`CScout.STAGES` covers the same six hubs, six facilities each, plus a booking paragraph per hub. A facility entry carries a name, a kind, a qualitative description of its stages, a notable line, an optional website and a `verified` flag; an unverified facility carries no website by rule, and gets a search link instead of a site button. Three of the thirty-six entries are unverified today — two in Toronto, one in Vancouver — and one verified entry has no website recorded, which also produces a search link.

These descriptions are abbreviated the same way the permit fields are: stage counts and square footages break off mid-figure, and the booking paragraphs break off mid-address. Where an entry looks incomplete it *is* incomplete. What survives intact across all six booking notes is the thing worth knowing: at the professional tier nobody publishes a rate card, and every one of these markets is direct quote, negotiated with the facility's own bookings team.

Sun times, and the timezone that decides them

Section 4 computes daylight for a pin. Pick a location from the dropdown to copy its latitude, longitude and recorded offset into the bar, or type them; add a date and press **Compute**. The maths is `tools/lib-sun.js` (`TSun`), the tested NOAA/Meeus implementation described in Chapter 19, accurate to about two minutes. A cruder second solar engine that once lived in `lib-scout.js` has been deleted, and `scripts/test_locations.mjs` asserts its absence.

You get sunrise, both golden-hour boundaries, sunset, civil dusk, daylight hours, the bearings the sun rises and sets on, and solar noon with its altitude. Where the sun does not cross the horizon at all, the panel says *midnight sun* or *polar night* instead of a time.

> **The clock is yours to supply. The code will not guess it.** `sunTimes()` returns UTC instants; `fmtLocal(ms, offsetMinutes)` renders them in whatever offset it is given, and given none it uses the offset of the machine you are reading on. The Scout Book therefore takes, in order: the number in the **UTC offset (min)** box, the `tzOffsetMin` recorded on the location, or — failing both — `tzOffsetFromLon()`, which is `round(lon / 15)` hours. That last is solar mean time: it knows nothing of political borders and nothing of daylight saving, so it can be a full hour wrong. The panel marks it ESTIMATED whenever it is in use, and that label is the thing to read before publishing a call time. Enter the location's real civil offset in minutes — `-420` for Los Angeles in August, `330` for a half-hour zone — and it is stored with `tzSource` set to `entered`. Nothing here fills that field in for you: the comments describe the offset arriving from the forecast service, but the only writer of `tzOffsetMin` on a location record is you, typing it. For scale, `scripts/test_sun.mjs` pins one LA sunset at 19:28 in the location's own offset and 02:28 in UTC.

Weather, and what leaves the device

`TSun.weatherUrl()` builds a request to `https://api.open-meteo.com/v1/forecast`. It sends the location's **latitude and longitude**, a start and end date, `timezone=auto`, and a fixed list of daily variables: weather code, maximum and minimum temperature, maximum precipitation probability, maximum 10 m wind speed. No key, no account, no production name; your browser calls that host directly, which is why `https://api.open-meteo.com` appears in the `connect-src` directive of `_headers`. A coordinate pair for a private house is still a disclosure.

Be precise about where it is made, because it is not this page. The Scout Book loads `lib-sun.js` but never calls `weatherUrl`: its sun panel is local arithmetic and issues no network request. The forecast lives in the Producer Suite — the day planner in `tools/sched-weather.js`, loaded only on `/producer/`, and the call-sheet forecast line. Both take their pin from `SB_ShootPlan_v1`, a separate coordinate entered there, and both capture `utc_offset_seconds` from the reply as that plan's civil offset, which Scout Book records never receive. A blocked or failed request is reported as a failure: it may not look like a clear day, nor like a date beyond the forecast window.

Set Design & The 3D Builder

A set plan is two drawings that must never disagree: the one the art department builds from, and the one the DP frames against. The Set Designer keeps them as a single record — the same feet, the same rotations, the same format — and stands one up from the other with no conversion step between.

> **How this chapter was produced.** Every statement below was established by reading the files it names, not by operating the application. Cinamate is under continuous development; where a sentence here and the running software disagree, believe the software and treat this page as stale.

What this is, and what it is not

The Set Designer at `sets/index.html` draws a stage in feet, places flats and dressing on it, puts real lenses at real marks, and stands the result up so a designer can see whether a doorway reads and a DP can see what a lens covers. It is not a CAD package: there is no dimension chain, no wall assembly, no tolerance and no structural calculation in `sets/lib-set.js` or `sets/lib-set3d.js`. A flat is a rectangular prism with a height; it has no studs. These are planning measurements, not construction drawings, and nothing leaving this module has been engineered.

The plan, and what the units are

Everything is measured in **feet** — the plan, the 3D geometry, both model exports. Neither module works in metres or in pixels; the only metric step is internal to the depth-of-field maths, which converts back before the number reaches you.

A new document opens with one plan, *Set 1 — Untitled*, 24 ft by 18 ft, editable in the right-hand inspector. The stored value is floored at 6 ft; the field advertises a 200 ft ceiling, but that belongs to the form control rather than the model, so a larger value typed past it is kept.

Snapping is a **half-foot grid** — a six-inch module — applied on every drop and drag. Dragging clamps an item's *centre* to the stage rectangle, so a ten-foot flat parked on the edge overhangs by five feet, which is what a wall on the edge of a stage does. Each plan carries a free-text *Scenes* field, tying the set to the scenes it serves.

The stencil catalog

Nineteen stencils are defined, each with a footprint in feet and a matching 3D profile giving it a height and a shape. Clicking a palette button drops the piece mid-stage; you then drag it into place.

PIECE	PLAN W×D FT	HEIGHT FT	3D SHAPE	PIECE	PLAN W×D FT	HEIGHT FT	3D SHAPE
Wall / flat	10 × 0.5	10	box	Rug	8 × 5	0.05	box
Door	3 × 0.5	6.75	opening	Plant	1.5 × 1.5	4	cylinder
Window	4 × 0.4	4, at 3 off floor	opening	Piano	5 × 6	3.3	box
Table	5 × 3	2.5	top + legs	Vehicle	15 × 6	5	box
Chair	1.6 × 1.6	3	seat + back	Green screen	12 × 0.5	12	box
Sofa	7 × 3	2.7	base, back, arms	Blocking mark	1.2 × 1.2	5.8	figure
Bed	6.5 × 5	2	box	Camera	1.6 × 1.6	5	body + lens
Desk	5 × 2.5	2.5	top + legs	Light	1.4 × 1.4	7	head + stand
Counter	8 × 2	3	box	Custom box	4 × 4	3	box
Shelf	3 × 1	6	box				

Every number there is a default. The inspector exposes W, D, rotation, *Height ft*, *Off floor ft* and a colour on any selected piece. A nonsensical height falls back to the profile default rather than producing a broken mesh, and an unrecognised stencil type resolves to *Custom box*.

Walls, flats and openings

A wall is a box, ten feet tall by default. Doors and windows are not solids: each is built as the frame *around* a hole — two jambs three-tenths of a foot wide, a header above the opening, a sill below it where the piece sits off the floor. A 6.75 ft door in a ten-foot wall gets a header and no sill; a 4 ft window raised 3 ft gets both. Those frames run to the tallest wall on the plan, or ten feet if that is taller, so raising one flat to sixteen feet raises every door frame with it.

Camera and lens

A Camera stencil carries a focal length in millimetres (35 mm by default) and an aperture (f/2.8), and the plan itself carries the **format** it is shot on. That one key answers every optical question the module asks — the cone on the plan, the frustum in the 3D view, the frame you look through — so the plan and the viewport cannot report different lenses. The format table lives once, in `tools/lib-media.js` as `TMedia.SENSORS`, and both set modules read it rather than keeping copies. Nine formats are offered; the default is `super35`.

KEY	FORMAT	MM	FRAME ASPECT	35 MM COVERS
super35	Super 35	24.9 × 18.7	1.33	39.2°
super35-17x9	Super 35 17:9	24.6 × 13.1	1.88	38.7°
fullframe	Full frame	36 × 24	1.50	54.4°
alexa-lf	ARRI LF	36.7 × 25.5	1.44	55.3°
alexa-65	ARRI 65	54.1 × 25.6	2.11	75.4°
red-vv	RED V-RAPTOR VV	40.96 × 21.6	1.90	60.7°
mft	Micro 4/3	17.3 × 13	1.33	27.8°
s16	Super 16	12.52 × 7.41	1.69	20.3°
iphone-main	Phone main cam	9.8 × 7.3	1.34	15.9°

Field of view is plain thin-lens trigonometry: twice the arctangent of half the sensor dimension over the focal length — horizontally against the sensor's width, vertically against its height. The inspector prints both to one decimal beside the format's name, because a coverage figure with no format stated next to it is a number nobody can check.

The frame's *shape* comes from the format and only from the format. In the 3D view the lens frame is letterboxed inside whatever shape the panel is; widening the browser window adds matte, never coverage.

> **A gap worth knowing about.** The frame aspect is the sensor's physical aspect. Nothing in these files implements an anamorphic squeeze or a delivery-aspect crop, so a 2.39:1 extraction cannot be dialled in — choose the format closest in shape and read the safe-area guides described below.

Depth of field

Depth of field is computed in feet. The circle of confusion is the frame diagonal divided by 1500 — the convention yielding roughly 0.029 mm on full frame and 0.021 mm on Super 35. Hyperfocal distance follows from focal length, aperture and that circle; at or past it the far limit is reported as infinity rather than as a large fiction.

What the camera is *focused on* defaults to the answer a 1st AC would give: the nearest Blocking mark the camera is pointed at — within sixty degrees of the lens axis, at least half a foot away, in front rather than behind. With no mark placed it falls back to 12 ft, and any camera may override it outright.

On the plan the sharp band is two arcs struck across the camera's cone, clamped at 60 ft so a deep stop does not run off the stage; the inspector prints the same limits in feet and inches. The 3D view reports the band in its caption but does not blur — there is no defocus rendering in the viewport.

> **Where the numbers stop.** Depth of field requires `tools/lib-media.js`. The Set Designer loads it; a page that loads the set geometry alone — the Props module does — gets fields of view but no sharp band, and the arcs simply do not appear.

Lighting

A Light stencil draws a throw on the plan: a fixed wedge of about forty degrees, nine feet long, aimed by the piece's rotation. It is a placement indicator, not a photometric model — there is no intensity, colour temperature, falloff or beam-angle field in the source. The distinction carries into three dimensions, where the viewport is lit by two fixed directional lights baked into its shader — a key from high front-left, a cooler fill from behind, plus ambient — chosen so every face stays distinguishable. Placing a Light puts a 7 ft stand-and-head model in the scene; it illuminates nothing, and the viewport casts no shadows.

The 3D builder

The 3D view is a second way of looking at the same items: nothing is converted and nothing duplicated, so an edit in one appears in the other immediately. Furniture is built as furniture rather than as crates — a table is a top on four legs, a chair adds a back, a sofa is a base with a back and two arms — which is what makes a plan legible at a glance.

The conventions are worth stating plainly, since they govern what an export looks like elsewhere:

- Units are feet.
- World axes are X right, **Y up**, Z toward the viewer — right-handed, matching OpenGL. A plan point (x, y) becomes world (x, 0, y), so the plan's depth axis is the world Z axis.
- An item's rotation is degrees clockwise on the plan, which is a rotation about the world Y axis, matching SVG's own `rotate()`. A camera at rotation 0 points up the page, which is −Z.
- Triangles wind **counter-clockwise seen from outside**, so a face normal points away from the solid it belongs to. Back-face culling is enabled in the renderer, which makes a winding mistake show up as a missing wall rather than as nothing at all.

The viewport draws a floor grid at one line per foot, brighter every ten, with the stage outline picked out, and draws every camera's frustum as wireframe so coverage reads without leaving free orbit. Drag orbits, shift-drag or right-drag pans, scroll dollies, a pinch zooms; clicking selects the same item a click on the plan would.

Looking through the lens

Look through puts the viewport at a camera's mark, at that camera's real lens on the plan's format — the thing a general-purpose modeller cannot do, because it has no idea what a 35 mm on Super 35 covers. While locked, orbit and zoom input is refused, the image is letterboxed to the format's aspect, and the frame carries the guides every monitor on the floor has: the frame edge, a 90% action-safe box, an 80% title-safe box and a centre cross. The caption names the camera, focal length, format, horizontal and vertical coverage, aperture and sharp band. Delete that camera, or switch plans, and the lock releases itself and says so rather than claiming to be a lens it no longer has.

> **If WebGL will not start.** The 3D panel reports that it is unavailable and the module carries on. The plan view and every export still work; nothing about the plan depends on the viewport.

Exports, and what each is for

Four buttons, all producing downloads named after the plan, lower-cased, with runs of non-alphanumeric characters becoming hyphens.

SVG — the plan

A vector top-down plan at 14 pixels to the foot, carrying the grid, every piece, camera cones with their depth-of-field arcs, light throws, the plan name and dimensions, the format's name, and a labelled 5 ft scale bar. It opens in Illustrator, Figma or a browser.

This file is **self-contained by design**. Because it is downloaded and opened elsewhere, every fill and stroke is written as a literal colour rather than inherited from the application's theme — no stylesheet travels with it and there is no theme to inherit from. That includes the background, which is the same deep navy the application uses, so sending the file straight to a printer lays down a full-bleed dark field; a light plan for paper wants recolouring in your editor rather than a setting here. The export also never carries your current selection highlight.

PNG — the call sheet

The same plan, rasterised through a canvas at twice that size, for dropping into a call sheet or an email.

OBJ — the modeller

Plain text, in feet, with the units stated in a header comment, and one named group per piece. Faces carry three or four one-based indices, and each face declares its own corners: vertices are *not* shared between faces, so a six-sided box exports twenty-four vertices rather than eight. An importer will therefore show every edge as hard — weld or merge-by-distance for a smooth mesh. There is no material library and no `usemtl`, so the per-item colours set in the inspector live in the viewport only and do not survive the export.

STL — previz and printing

ASCII STL, one solid, every facet a triangle with its normal declared explicitly. Facets with no area are dropped rather than written, because a zero-length normal is not a direction and a slicer will believe whatever it is told.

> **STL declares no units.** The format has no mechanism to. The numbers written are feet, and most slicers read STL as millimetres, so a ten-foot flat arrives as a ten-millimetre one. Scale on import — by 304.8 if the receiving tool works in millimetres.

Names are filtered on the way out

Every name written into an OBJ or an STL passes through a slug filter: any run of characters outside `A–Z a–z 0–9 _ -` becomes a single underscore, and the result is truncated to 60 characters. A set called *Kitchen — north wall* is written into the file as `Kitchen_north_wall`. This is not cosmetic tidying — a newline in a name would otherwise end an OBJ comment and let the rest be read as geometry.

One thing to expect rather than discover: an OBJ group name is built from the piece's *internal identifier and its stencil type* — something of the form `i7k2m4a_wall` — not from the label typed in the inspector. Labels appear on the plan and in the SVG; they do not reach the model exports at all. If the receiving artist needs to know which flat is which, the SVG plan is the document that tells them.

What a receiving modeller should expect

Both exports wind counter-clockwise from outside, so face normals point outward and a default single-sided material renders correctly with back-face culling on. The suite pins that rather than trusting it: a box top is measured as pointing exactly +Y and its bottom exactly −Y, the STL is checked to declare `facet normal 0 1 0` and `facet normal 0 -1 0` for them, and every shape in the catalog is checked by the divergence theorem to enclose a positive volume, which an inside-out mesh cannot. Degenerate geometry is refused at source too: faces repeating a corner are cleaned before writing, cap fans go out as genuine triangles rather than quads with a doubled corner, and cylinders are capped at both ends, so a shape arrives as a closed solid a slicer will accept.

Where plans live

The document — every plan, every piece — is stored in this browser under the key `SB_SetDesign_v1`, written after each edit. It is local to the browser and the machine, and there is no version history.

> **Deletions are immediate and final.** Deleting a set plan asks for confirmation once, then removes it and everything on it. Removing a piece — by the inspector's button, or by Delete or Backspace while a piece is selected and no text field has focus — asks nothing at all and cannot be undone. Export anything you would mind losing before clearing a browser's site data, which takes the whole document with it.

What the suites hold in place

Two node suites cover this module without a browser: `scripts/test_set.mjs`, 69 assertions for the plan model, snapping, fields of view, depth of field and the SVG; and `scripts/test_set3d.mjs`, 132 for geometry, matrices, orbit, picking, winding and both exports. The one worth knowing as an operator is the agreement assertion: the plan, the 3D frustum and the shared format table are each asked the same lens question at 18, 25, 35, 50, 85 and 100 mm, and must answer within a twentieth of a degree of one another. Changing a format in one module fails both suites until the other follows.

Props & Wardrobe

Two departments, one habit of mind. Break the script into things; put a number on each thing; find someone who has one; and keep a record of what it looked like on the day, so the next shot matches. Cinamate builds both to the same pattern, deliberately.

> **On the accuracy of this chapter.** Everything here was established by reading the files it names, not by operating the software. Where this page and the running platform disagree, the software is the authority and this page is stale.

The props breakdown

The Props Department is `props/index.html` over the engine `props/lib-props.js` (`CProps`), stored in `SB_Props_v1`. *Rebuild from script* reads the working screenplay from `SB_Timeline_v1.scriptText` — under about forty characters it declines — splits it through the one scene model, `js/lib-scenes.js`, and runs a fixed lexicon of **191 terms across nine categories** over every scene body.

Three consequences. The lexicon is closed: a thing it does not name is not found, and goes in through + *Add prop*. Rows are deduplicated by term, so one row covers every appearance of that word and quantity starts at 1. And the match runs over the whole scene body, dialogue included — a gun somebody merely talks about is still listed. Scenes are recorded by printed identity, so an A-scene stays an A-scene.

> **Rebuild replaces, it does not merge.** Only rows you created with + *Add prop* survive. Every automatically found row is discarded and regenerated, taking its quoted price, its hero and period flags and any hand-entered dimensions with it. Rebuild after a script revision, not to refresh a list you have priced.

What a prop costs

Pricing uses the industry ten-per-cent rule: a week's rental is about a tenth of replacement value. That value, held per category, carries the whole estimate.

CATEGORY	VALUE	PRICED AS
Weapons & armory	$1,200	weekly; adds a licensed armorer
Picture vehicles	$20,000	$400 per shoot day
Furniture	$900	weekly
Electronics & tech	$450	weekly
Set dressing	$180	weekly
Hand props	$75	weekly
Greens & florals	$1,200	weekly
Food & consumables	$60	bought, $60 per shoot day
Specialty / oversize	$3,500	weekly

A *hero* prop triples that value, being dressed to close-up specification; *period* multiplies it by 1.6. The first rental week is charged in full, later weeks at 75%. Buying wins once renting passes 60% of the purchase price, assuming roughly 40% recovered at wrap — and since both sides scale with the same value, **every category priced by the ten-per-cent rule flips from rent to buy at eight rental weeks**, hero and period included. Vehicles bill twice their scene count in days, capped at the shoot length, and consumables are bought per shoot day, so neither reaches that break-even. Weapons add an armorer at **$650 a day**, once per production rather than per weapon, for three days on any schedule of three or more. Ten per cent contingency sits on top.

> **A gap, stated plainly.** The engine honours a replacement value per item, but the table offers no field for it — the editable columns are name, category, quantity, plan, hero, period, quoted price and size. Reading the source, there is no route in this page to price one prop away from its category default. Use the quoted column for a real figure.

Each row also carries a size in feet: the size button draws the prop beside a six-foot figure and says whether it passes a standard doorway, and → *Set* places it on the active plan in `SB_SetDesign_v1` at true size.

Sourcing, and the quote request

The directory holds twenty production hubs, twelve named houses across six of them, the rest recognised but empty. **No directory entry carries a telephone number.** Numbers arrive only from live map data, from the research service on your generation machine, or from your own typing, and what you type is saved against the production. *Use Advisor location* takes the jurisdiction you modelled for tax incentives from `SB_Budget_v1` and maps it to the nearest hub; the lookup geocodes that city through OpenStreetMap's Nominatim, searches Overpass within 40 km, and merges what it finds onto the directory so a matching listing fills in a phone rather than duplicating the card. Results cache for a week. Confirm every detail on the phone before quoting from it.

Section 3 assembles a quote request naming the production, dates, shooting area and every prop with quantity, category and rent-or-buy intention, asking for weekly rates, damage waiver and delivery. The email button truncates at 1,800 characters, so send a long list by copy and paste.

Wardrobe: characters, looks and pieces

Wardrobe is `wardrobe/index.html` over `wardrobe/lib-ward.js` (`CWard`), stored in `SB_Wardrobe_v1`. Seeding reads dialogue cues — a short all-capitals line, suffixes such as (V.O.) and (CONT'D) folded together, followed by real dialogue. A *look* belongs to a character, lists the scenes it is worn in, and holds sizes, notes and pieces. A piece has an item, a source — `buy`, `rent`, `build`, `cast-own` — an estimate and, later, an actual. The actual sits *beside* the estimate, never on top of it: overwriting an estimate with an invoice destroys the only record of what the department thought.

> **Two things that remove data.** Deleting a look also deletes its continuity photographs from this browser, without confirmation. And a character chip seeds a look from the legacy numeric scene list, which cannot tell 4A from 4 — after seeding, retype the scene box, which does accept A-scenes and ranges (`1, 4A, 6-9`).

What the department learns from an invoice

This is the part of both modules worth understanding properly. A department types an estimate; weeks later an invoice arrives for a different number; and until this loop existed the second number had nowhere to go. The reasoning is written out at `wardrobe/lib-ward.js` lines 169–206, copying `recordQuote` in `props/lib-props.js`. Three rules separate calibration from self-flattery.

It learns from the raw estimate, never the calibrated one. Feeding the corrected figure back in would teach the multiplier from its own output and compound it: the number would drift and always agree with itself. What reaches the learning layer is what you typed.

It reports the evidence being applied. Every priced piece carries `learnedN` — how many past invoices stand behind the multiplier in front of you. A correction resting on two invoices is never shown as though it rested on fifty.

Below the threshold it says so. Two invoices is the minimum. Under it the multiplier is exactly 1 and the page reads *not calibrated · 1 of 2 invoice(s) needed — estimates for this source are exactly what you typed*, rather than dressing a guess in arithmetic. Note the phrasing: evidence that *exists* is still reported while it is not yet being *used*.

The account is chosen so the learning means something. Wardrobe learns per *source*: a built garment overruns for different reasons than a rental does, and averaging them teaches nothing about either. Props learns per *category*, under `props:handprop` and its siblings. Both write to `CIN_Learn_v1`, outside the per-project `SB_` namespace on purpose, so what one production teaches survives into the next.

> **Nothing is learned until the film is wrapped.** The learning layer refuses every observation unless the active project is marked wrapped in Projects. Mid-shoot, an actual is only what has been invoiced so far against a whole-picture estimate — a department that has spent 12% in week two looks like an 88% underrun, and folding that in would drag every future estimate down. An invoice typed during production is therefore recorded on the piece and teaches nothing. A line also teaches once only: re-typing a corrected invoice against the same item does not re-teach it.

Where evidence exists, the multiplier is a ratio of actual to estimate, weighted towards recent films and clamped so no single catastrophe can double or halve an account. Read the size of a correction as how much correcting has been necessary, never as accuracy; the platform keeps a separate walk-forward record for that question, and it can

come back negative. An operator holding these three rules will believe the number when it is earned and disbelieve it when it is not.

Story days, and where they are uncertain

A **story day** is a day in the life of the story, not a day of the shoot. Scene 4A and scene 22 may be the same story day, in which case the shirt, the cut lip and the hair must match between them, though one shoots in week one and the other in week three. No scene record holds this: a slugline gives INT or EXT and a time of day, nothing more. `js/lib-storyday.js` derives it, comparing each scene against the last *printed* time of day so a run of CONTINUOUS headings does not lose the clock. Every boundary carries a confidence.

CERTAIN — THE SCRIPT SAID SO	UNCERTAIN — THE DERIVATION HAD TO CHOOSE
CONTINUOUS, SAME TIME, LATER, MOMENTS LATER	DAY after DAY: nothing says whether a night passed
NEXT DAY, THREE DAYS LATER, THE DAY AFTER	NIGHT after NIGHT: nothing says it is the same night
An explicit stamp on the heading — DAY 4	DAY into NIGHT: read as the evening of the same day
NIGHT giving way to DAY — the sun came up	A heading with no time of day printed at all
The first scene of the script opens day 1	Flashback or dream: a new day, but which one is unsaid

Uncertain boundaries default to *the same story day*, the conservative reading: it makes the department match more often, and an unnecessary match costs a photograph while a missed one costs a reshoot. Every uncertain scene is listed rather than hidden, and the page states the tally. Walk that list with the script supervisor. A hand-set day always wins, is marked as set by hand, and still shows what it overruled.

> **A wrong story day is a continuity error on screen.** Treat the derivation as a first pass. `SB_StoryDays_v1` stores only your decisions — overrides and day names — and the derivation is recomputed from the script every time, so a rewrite never leaves stale days pretending to be current. *Clear all overrides* discards every hand-set day at once and cannot be undone.

The costume plot, changes and doubles

Section 3 is character against scene, grouped by story day and cross-referenced to the shooting order from `SB_ScheduleBoard_v1`. The board identifies a strip by heading and ordinal, never by printed scene number, so the join is made here — and a strip matching no scene is reported rather than quietly dropped.

Changes are numbered per character in story order, which is what a supervisor calls out: MAGGIE #1, #2, #3. A look that returns keeps its number, because "back into change 2" is the instruction. Each is flagged: *must match* when worn in more than one scene; *shot out of story order*, the classic trap where a garment's torn and bloodied state is arrived at on story day 2 and photographed on shooting day 9; *uncertain* where the story day is unsettled. Quick changes are measured *inside* the story day, where the clock runs, and the flag says whether both scenes shoot on the same day — a change on the floor with a dresser standing by, or a change on the truck. Two looks in one scene is a conflict; a look pinned to a scene the screenplay does not contain is named as such.

Doubles come from the script. Six hazards are read per scene — blood, rain, mud, tearing, fights, water. A look meeting one gets three multiples (one to wear, one to wreck, one spare), plus one per further hazard scene, capped at six. The advice says outright that it is an estimate: confirm with stunts and SFX.

Continuity photographs

Photographs attach to a look, are scaled so the longest edge is at most 1,024 pixels, stored as JPEG, and stamped with the date and the active project. The bytes go into IndexedDB — database `cinamate_wardrobe`, store `photos`.

> **The vault does not carry your photographs.** The project vault snapshots `localStorage` only. The photo *references* travel with the project and into every archive; the *bytes* stay in this browser. Switching projects strands them, and an archive restored on a second machine shows empty frames. The scan counts exactly that — yours, another production's, orphans no project points at, and references whose bytes are not on this device. *Export photo pack* is the only way to move them; *Purge orphans* deletes bytes permanently.

Commit total to Money Room raises a purchase order against account 10000 for the wardrobe total, its description recording whether that figure is calibrated or an uncalibrated estimate. All three engines are covered by node suites that run without a browser: `scripts/test_props.mjs`, `scripts/test_wardrobe.mjs` and `scripts/test_storyday.mjs`.

On Set — Production, Dailies & Today

This is the part of the platform read standing up, outdoors, with something else already in your hands. Three pages carry the shooting day: `/dailies/` logs the takes, `/today/` is the day in your pocket, and `/production/` turns both into paperwork — the only record of what the day actually did, which is a different document from the one that said what it would do.

> **On the accuracy of this chapter.** It was written by reading Cinamate's source, not by working a shooting day with it. The platform is under development while this manual is being printed. Where the two disagree, the software is the authority.

One day, one record

A day means three things here — an index on the stripboard, a calendar date in the day planner, a date and unit on the logger — and until they were joined, none could find the others. The join is `SB_ShootDays_v1`, in `js/lib-shootdays.js`, holding `dayIdx`, `date` as `YYYY-MM-DD`, `unit`, the day's `sceneIds`, `wrapped` and `pinned`.

Two points of it matter on the floor. First, **a day is keyed on date *and* unit**, never the date alone: main unit and a splinter both shooting the 14th are two days' work, two call sheets and two records, and the date alone let the second silently overwrite the first. Second, **a take joins to a day on the date** and on nothing else — matched by exact string equality, across both take stores. A take carrying no date is on no report at all, deliberately, because guessing that it belongs to today is the bug this replaced; undated takes are counted separately and said out loud.

That date is the production's **local** calendar date. Wrap at 23:50 on the 7th belongs to the 7th — the day the crew worked, the day on the call sheet.

Logging a take

`/dailies/` runs in the order the day happens. Section 1 sets the shoot day and the unit (`MAIN`, `2ND`, `SPLINTER`) and shows which record that date resolves to, or *not on the stripboard yet*. Touching either control writes the record and marks it `pinned`, so a date a human stated is not recomputed by a later rebuild.

The take form is deliberately large-target: scene, slate, take, an A/B camera pair, lens, sound roll, TC in, an NG reason from a fixed list, and a note. Type a scene number and the slate and take fill themselves in from what you have already shot — same scene, same slate, next take number; a fresh scene starts at slate `A`, take 1. **NEW SETUP** bumps the setup letter the way a 2nd AC chalks it: 12A to 12B, 12Z to 12AA, the letters being bijective base-26. **+ TAKE** logs it; **● CIRCLE LAST** circles or un-circles what you just logged. On a phone those two sit in a fixed bar at the foot of the screen.

> **The ✕ on a take row deletes it at once.** No confirmation, no undo: the row leaves `SB_Dailies_v1` with its circle. Note also that the table shows only this page's own takes — rows logged in Tools → Slate live in `SB_TakeLog_v1` and count towards every report and rate below, but cannot be edited here.

The circle

A circled take is the print order, written in two alphabets. This page stores a real boolean; Tools → Slate stores the `<select>` display string `Circled ⭕` — the word *Circled* followed by the red-circle emoji — in a field called `grade`. `js/lib-shootdays.js` translates the second into the first once, so every consumer gets one answer. **`Good` is not a circle**: the grade list offers both, so an AD who chose *Good* chose it on purpose. Section 5 gives the circle rate overall and per day, and **Send picks to Editor** writes the circled takes to `SB_DailiesPicks_v1` as a pull list, replacing whatever was there.

Two logs, two clocks

This page stamps a take with the local date; `tools/tools-core.js` stamps `SB_TakeLog_v1` in UTC. When the two dates differ — west of Greenwich, most of the evening — takes logged on the phone slate file under tomorrow and will not join tonight's day. `/dailies/` prints a warning saying so; until the writers agree, log late takes here.

Scheduled and not shot

Section 4 answers the question actually asked at wrap: *of the scenes scheduled for today, which did we not get?* The stripboard says what was scheduled on a day index, the shoot-day record turns that index into a date, the take logs say what was shot on it. It needs a stripboard and a screenplay in this browser, and names whichever is missing.

A day is judged only once it is over — marked **wrapped**, or its date has passed; a scene scheduled for next Tuesday is not a gap. Results are separated by kind, because they are different facts: scheduled and never shot anywhere, with the outstanding pages in eighths; missed on the day and picked up since; shot on a day it was not scheduled for; and the joins that failed — strips matching no scene, takes against a scene number the screenplay lacks, scenes on no strip at all. **Copy the gap list** puts all of it on the clipboard for the production report.

Calling the wrap

Marking a day wrapped is the most consequential thing done on these pages. `wrapped` is the flag the schedule's pace model gates on: the board ships a planning default of 4.5 pages per day and replaces it with the median of what your *wrapped* days actually achieved, once three of them exist. A production that never wraps a day reads the shipped default for forty days and learns nothing from itself — and the pace chip says exactly that, in the words *nothing learned yet (no wrapped days)*. Wrapping also turns on the gap list above.

The control is on `/today/`, beside the day selector: ◉ **Wrap day**, becoming ↺ **Re-open day** once set. It asks first, names the day it is about to wrap, and is reversible through the same call — a day wrapped by mistake would otherwise teach the schedule a pace nobody worked. Two things established by reading: this page is the **only** place in the shipped tree that sets the flag, and `scripts/test_wrapgate.mjs` exists to catch it if that stops being true; and the stripboard *displays* a `WRAPPED` chip but does not toggle one, whatever its own comment says. Wrap the day on set, on the day.

> **The wrap button only offers days the stripboard knows.** `/today/` builds its day selector from the strips on `SB_ScheduleBoard_v1`. With no board in this browser there is no selector and no wrap control, and a day created only by `/dailies/` — a date the board has never heard of — falls outside the selector's range and cannot be wrapped from here.

What `/today/` shows

One day, phone-sized, found by lookup rather than by guessing at a date string. If today is a shoot day you get today; otherwise you get the next day that has not wrapped, headed `NOT TODAY` so the two cannot be confused. A wrapped day is headed `WRAPPED`.

The sheet carries the production name; the call time typed on the stripboard's call-sheet panel; day number, weekday, date and unit; the page total in eighths; the day's scenes with day/night and page count; the cast working; the sets parsed from the sluglines; the nearest hospital from the Scout Book; the safety checklist for today's scene numbers; and the call sheet's notes. Where the hospital is not on file it says so and points at the Scout Book, because it belongs on every call sheet.

The daily production report

Production Office → **Daily Report** assembles a DPR for one date from what the platform already holds; nothing on it is retyped. Pick a date — it opens on today if today is a shoot day — add notes, and build. The report heads itself with date, day number and unit, or says *not a scheduled shoot day* rather than quietly reporting the whole schedule as one day's work. It then gives the scenes scheduled on that day's strips and how many were shot, the scenes NOT shot by number, every scene covered, the take count with the circled count beside it, the crew timecard count and hot costs. 🖨 **Print** and ⬇ **.txt** are the outputs.

Two figures are wider than the day, and say so in their wording rather than in their arithmetic: hot costs are every posting to date, and a timecard carrying *no* date is counted on every report you build. Where the take logs hold undated takes, the report prints the count and tells you to date them.

Continuity, camera and sound

The Continuity pane keeps one row per scene and setup in `SB_Continuity_v1` — circled take, screen direction, wardrobe and props, matching notes. Beside it the Camera and Sound panes keep the classic per-roll registers, down to lens, stop, filter, timecode and mics. `/dailies/` also exports a camera report and a sound report for any one day, columned like the paper ones with circled takes marked ●. Both print the same caution: cross-check them against the camera assistant's and the sound mixer's own sheets before lab or DIT turnover.

Cards and footage

Footage is described here, never held here: a take records its camera, lens, sound roll and TC in, and the registers record the roll. The one place the platform touches media itself is Tools → **Offload Verification**. Pick a card's files and each is SHA-256 hashed in this browser, producing a manifest to keep with the media; load that manifest later, re-pick the copied files, and each comes back ok, changed, missing or extra — a clean result reported as bit-perfect. Nothing is uploaded; the hashing happens on the device. The manifest is Cinamate's own XML, rooted at

`<cinamatemanifest>`: MHL-shaped, but not an ASC MHL file, so do not hand it to a DIT expecting their tool to read it.

Sides

> **What this platform calls sides is not a redacted side.** Both builders were read for this chapter, and both emit the *complete scene*: the slugline followed by the entire body — every other character's dialogue, and all the action — for each scene they select. Nothing is withheld or blacked out. They differ only in which scenes those are. The Casting Office (`casting/lib-castdesk.js`) takes the scenes in which the character has a dialogue cue — audition sides, the scenes a performer would actually perform. The Production Office (`production/lib-prod.js`) takes the scenes the character *appears* in, whether by cue or by name in an action line, because an actor who crosses a room without speaking is still called that day; the name is matched as a whole word, so a character called AL is no longer pulled into every scene containing CALL. Either way, what comes out of **⬇ .txt** is an unreleased extract of the screenplay, and sent unedited to a performer's representative it sends those pages in full. Cut what the room should not see before it leaves your hands — as the Casting Office's own header says, trim for the room and verify against the current draft before sending.

What these pages do not do

- **They do not upload footage anywhere.** Neither `/dailies/` nor `/today/` sends anything to a server: once loaded, they read and write this device's own storage, and the take logs, the reports and the DPR are all assembled in the browser.
- **They notify no one.** Nothing emails a call sheet, texts a unit or tells the office a day has wrapped. Every output is a printout, a `.txt` download or the clipboard, and it moves because a person moves it.
- **None of it replaces the script supervisor's own notes.** It is a parallel record, valuable for the joins it makes across the schedule and the take logs — and its own exports say as much.
- **They are not offline apps.** The service worker caches only the public shell, by an allow-list these pages are not on, so nothing serves them without a connection. The data they read is local to the browser that logged it: a take logged on one phone is on that phone.

The Editor I — Cutting

Everything in this chapter happens in your own browser. The bin, the sequence, the trims and the playhead are held on the machine in front of you; no frame is uploaded in order to cut it, and nothing here needs a network. That is the Editor's great convenience and its one real hazard.

> **On the accuracy of this chapter.** It was written by reading the Editor's source, not by operating a finished product. Cinamate is still being built. Where this page and the running software disagree, believe the software, and treat the disagreement as a fault in the manual.

Two editors carry the name

The full Editor is `/editor/`, whose page loads the cut engine `editor/lib-cut.js` (publishing `window.CCut`), the MP4 writer `editor/lib-mp4.js` and the cutting room itself, `editor/cut-ui.js`. Three tracks, titles, undo, a persistent bin. That is what this chapter documents. The smaller editor in `editor/timeline-engine.js` is mounted inside the Studio at `/timeline/`, in the collapsible *Editor* panel: one video lane, no titles, no undo, its own storage key. Work done in one does not appear in the other.

> **Destructive.** In the Studio's Editor panel, *Sync clips* and *Pro Cut* both begin by emptying that panel's bin and edit track and rebuilding them from the Studio's clip list. There is no undo there, so trims made by hand are gone. The full Editor is unaffected — it does not share that state.

Getting media in

The bin fills from two places, and the difference matters more than anything else on the page.

Load Studio clips reads `SB_Timeline_v1`, takes every clip that has a `videoUrl`, and adds one row each, named `SC01`, `SC02` and so on with the clip's label appended. The row records the *address* of the render; the bytes are not copied.

Import files opens a picker accepting `video/*` and `audio/*`. Each chosen file is written into IndexedDB before anything else happens, then added to the bin under an id of the form `f_c…`. The bytes are copied, and they are yours.

Either way the source is then probed by a detached element for duration and, for picture, dimensions and a thumbnail. A source that refuses a pixel read — a cross-origin render, typically — yields no thumbnail; one that errors outright is marked `missing`.

Where the media actually lives

Imported files go into IndexedDB: database `cinamate_cut`, version 1, object store `media`, out-of-line keys — the key is the bin id, the value is the raw `Blob`. On the next visit each flagged row is read back and given a fresh

object URL; a row whose blob has gone is marked missing and says so, worded differently for a file than for a link.

The cut itself does *not* live there. It is written to local storage under `SB_Cut_v1`: the project object, a stripped copy of the bin (id, name, kind, duration, dimensions, origin and the `idb` flag — the object URL is deliberately blanked for rows held in IndexedDB, since an object URL dies with the tab), and a note of the last export. A production's writing and its pictures therefore sit in two different stores, and they travel differently.

What that means for backup

The vault knows about the store. `projects/lib-vault.js` carries a list called `BLOB_DBS`, and `cinamate_cut/media` is on it, labelled *cutting-room source*. A row is claimed for a production by reference: the vault reads `SB_Cut_v1`, walks its `bin` list, and takes the id of every row whose `idb` flag is set. Two consequences follow.

- An **imported file** travels. Projects → Export backup packs its bytes into the `.cinamate` archive; Import backup writes them back into `cinamate_cut/media` on the receiving machine.
- A **Studio clip** does not. It carries no `idb` flag because it is a link and there are no bytes to carry. It restores as an address, and whether that address still resolves is not a question about your backup.

Two ceilings apply. The vault's media adapter will not read any single record larger than 16 MB into memory; such a record is excluded from the archive's `blobs`, listed under `blobsMissing` instead, and the export message states how many files "had no bytes on this machine and are NOT in it". The studio cloud is far tighter — 900 KB per record, 48 MB per production — and refuses the whole push rather than truncating it. A source master is not going to the cloud.

> **An older cloud version comes back without its media.** The cloud keeps eight superseded copies of a production in a rotating ring, but the ring holds the written record only. Media parts live under separate keys and are never versioned — the restore endpoint says so in its own reply, `mediaVersioned: false`. Restore last Tuesday's version and you get last Tuesday's cut list pointing at whatever media is currently on file; bin rows may come back missing. If a cut matters, export a `.cinamate` backup. That is the only path carrying picture and cut list together.

The bin

Rows show a thumbnail, a name and a duration, with Studio rows marked · *studio*. Double-click a row to append it to the timeline, or drag it onto a track. A missing source keeps its row, marked, and refuses to be added — a cut is a list of references, and a row that quietly disappeared would leave the sequence pointing at nothing while looking shorter than it is. Nothing in the source removes a row: within one cut, the bin only grows.

The timeline model

A cut is a plain object with a name, a frame rate, a frame size and three lists.

```
video[]   {id, srcId, label, in, out, speed, trans:{type,dur}, color}
titles[]  {id, text, sub, start, dur, pos, size}
audio[]   {id, srcId, label, start, in, out, gain}
```

`in` and `out` are positions in the *source*, in seconds. A video clip does not record where it sits on the timeline, and that omission is the design.

Effective duration, and the ripple

A clip's length on the timeline is not `out − in`. It is

```
effDur(c) = max(0, (c.out − c.in) / (c.speed || 1))
```

so a four-second selection played at 2× occupies two seconds. Start times come from walking the video list and accumulating those lengths: clip *n* begins at the sum of every effective duration before it. Nothing stores a start time, so nothing can disagree with one. Trim the head of clip 1 and every clip after it moves. That is the ripple, and it is not a mode you can switch off.

The title and audio tracks work the other way: each entry carries an absolute `start` and stays where it is put. This is the trap. Retrim the picture and your titles and music do *not* follow — a title cued to a line of dialogue drifts the moment you tighten anything upstream of it, and only your eye will catch it. Overall length is the longest of the three.

Transitions

A transition belongs to the clip it plays *into*. `cut` has no duration; `crossfade` holds the outgoing clip's last frame over the incoming one at falling opacity; `fadeblack` is split down the middle, bringing the incoming clip up from black over its first half and darkening the outgoing clip's tail over the same half-length, so a dip reads across the join rather than only after it. The inspector allows 0 to 3 seconds, defaulting to 0.5 s. Titles fade over a fixed 0.3 s and cannot be changed.

Trimming, splitting, rippling

Every clip on every track has a handle at each end. A drag converts the pointer's travel to seconds by dividing by the current zoom, then multiplies by the clip's speed, because a pixel of timeline is `speed` seconds of source. Dragging the left handle of an audio block also moves its `start`, so it trims from the head instead of sliding. Each drag is clamped as it goes: `in` is forced to zero or above, `out` is clamped to the source's known duration, and a floor of 0.1 s holds between them — a clip cannot be trimmed to nothing. Where the source duration is unknown, the upper clamp is not applied.

Split (the button, or `S`) cuts the video clip under the playhead in two: the first half takes the playhead's source time as its new `out`, and a second clip begins there with the same source, label and speed and a plain cut into it. A split refuses within 0.05 s of either end of a clip, and leaves total duration unchanged — which the checks in `scripts/test_cut.mjs` assert.

Reordering is a drag of a clip onto another in the same track. On the video track that reorders the list and the ripple recomputes; on the other two it sets a new absolute start.

> **Destructive.** *Remove clip* in the inspector, and the `Delete` and `Backspace` keys, delete the selected item immediately with no confirmation. Each takes an undo snapshot first, so `Ctrl+Z` brings it back — provided you have not since spent the sixty-step history.

Frame rate, and why timecode is counted in frames

The rate is a property of the *cut*, not of the export dialogue. The picker writes `project.fps` the moment it changes, and the ruler, the timecode readout and every turnover file are counted from that one number. One function, `normFps()`, decides what qualifies as a rate: anything finite and positive is accepted, 23.976 included, and anything else becomes 24. One default, in one place, so a cut that has never had a rate written to it cannot pick up a different assumption in each exporter. The picker offers 24 and 30; a saved cut carrying a rate this build has no option for is corrected to match the picker rather than the two silently disagreeing, so check the rate when you open an old cut.

The important part is the arithmetic. Timecode comes from `tc(seconds, fps)`, in this order:

```
rate   = max(1, round(fps))         // non-drop counts the NOMINAL rate
totalF = max(0, round(sec × rate))  // ROUND ONCE, INTO FRAMES
f = totalF % rate
s = floor(totalF / rate) % 60
m = floor(totalF / (rate × 60)) % 60
h = floor(totalF / (rate × 3600))
```

Every field is derived from a single frame count, and nothing is rounded to whole seconds on the way. That is the whole point. Round to seconds first and every sub-second trim is discarded before the frames field is even computed: a half-second handle becomes zero, a twelve-frame overlap becomes nothing, and a run of small trims drifts silently against the picture it is meant to describe. The test suite pins this — `tc(0.5, 24)` must be `00:00:00:12` and `tc(1/30, 30)` must be `00:00:00:01`. The same discipline governs the interchange formats, where each lane keeps its cursor in frames and rounds once rather than accumulating rounded seconds. A fractional rate still prints against the nominal integer: at 23.976 the frames field runs 00–23 and one hour of media reads `01:00:00:00`. This is non-drop timecode, labelled as such.

> **What frames-based timecode does not do.** The model stores `in`, `out` and `start` as floating-point seconds, and nothing in the source snaps them to frame boundaries. A trim handle can land at 4.3708 s and will stay there. The frame count is computed at the moment a number is displayed or written out, so what you read is always frame-accurate to the rate — but two operations that round separately can land a frame apart. If a cut must be frame-exact, set the rate before you trim and type figures into the inspector rather than dragging them.

Audio

Audio arrives two ways. A video clip brings its own — the preview element is unmuted, so picture and production sound play together. An audio file imported into the bin lands on the A1 track as a free-positioned block with its own in and out points and a gain from 0 to 1. Which track an item goes to is decided by the bin row, not by where you drop it: a source is classified once, at import, by MIME type, and an audio row always lands on A1, a picture row always on V1, wherever the pointer released it.

In playback the A1 blocks are driven by their own hidden media elements, started when the playhead enters the block and nudged back if they drift more than a quarter of a second. Waveforms are drawn lazily — the source is fetched, decoded, reduced to 240 peak buckets and cached — and a source that cannot be decoded is marked as having no waveform and not attempted again that session. A blank strip therefore means the decode failed, not that the audio is silent.

Undo

Undo is a snapshot stack. Before each mutating action the whole project is serialised to JSON and pushed; the stack holds sixty entries and discards the oldest beyond that. Redo is cleared whenever a new snapshot is taken. A field committed without actually changing anything has its snapshot popped again, so the history does not fill with no-ops.

Two limits are worth stating plainly. Undo restores the *project* — the three tracks, the name, the rate. It does not restore the bin, so it will not bring back a source, and it clears the current selection. And it is held in memory only: it does not survive a reload, however recent the change. `Ctrl+Z` and `Ctrl+Y` (or the Command key) are wired, as are the two toolbar buttons. `Ctrl+Shift+Z` is not.

What is saved, and when

There is no Save button, and none is needed. Every action that changes the cut writes `SB_Cut_v1` immediately: adding to the bin or a track, a trim released, a reorder dropped, a split, a title added, an inspector field committed, a rename, a rate change, an undo, a redo. A write that fails — a full quota, most often — is swallowed silently, and that is the one case where the page will not tell you something went wrong.

What is *not* saved: the playhead, the zoom, the selection, the undo history and the object URLs. All are rebuilt from zero on the next load, with the playhead at the start.

What the Editor is not

It is not a finishing tool. The per-clip colour controls are four sliders — exposure, contrast, saturation, warmth — compiled into a canvas filter of `brightness`, `contrast`, `saturate` and either `sepia` or `hue-rotate`. No curves, no per-channel control, no windows, no keyframes, no scopes; a setting applies flat across the whole clip for its whole length.

It does not conform to a master. There is no relink step and no concept of a camera original. What the Editor previews and what it renders are the same files that are in the bin: Studio renders at whatever size they were generated, or the files you imported. Turning over to a finishing suite is what the interchange formats are for, and they are the next chapter's subject.

It is not a substitute for a colour session. The export path is an 8-bit H.264 encode off a canvas, tagged Rec. 709 limited range — no log pipeline, no LUT, no wide-gamut or high-dynamic-range handling anywhere in it. Grade here to make an assembly readable, then grade properly elsewhere.

Keyboard

Shortcuts are ignored while a field has focus, so a clip name containing the letter *s* can be typed without splitting anything.

KEY	ACTION
`Space`	Play or pause at 1×.
`K`	Play or pause at 1×, the same as Space.
`L`	Play; press again to shuttle 2×, then 4×, then back to 1×.
`J`	Step the playhead back one second. It does not shuttle in reverse.
`S`	Split the video clip under the playhead.
`Delete` / `Backspace`	Remove the selected clip, title or audio block.
`Ctrl`/`Cmd` + `Z`	Undo.
`Ctrl`/`Cmd` + `Y`	Redo.

The zoom slider runs from 20 to 240 pixels per second and opens at 70. The ruler is clickable along its length to place the playhead; its labels show minutes and seconds, while the transport readout carries full timecode from hours to frames. Review notes sent from the Screening Room stand on the ruler as markers.

The bin's Assistant buttons — rough cut, tighten, cut to beats — and every export and delivery format are the subject of the next chapter.

Tools & Utilities

Every production accumulates arithmetic that belongs to nobody in particular. What time does the light go. What does a fourteen-hour day cost once the meal was late. Is the copy off the card actually the same file. Who has the call sheet, and did they say so. The Tools module is where that arithmetic lives — nineteen tabs behind one rail at `/tools/`, sharing four calculation engines and one table component between them.

> **On the accuracy of this chapter.** It was compiled by reading the source files, not by operating the software. Cinamate changes week to week; if a screen contradicts a sentence here, the screen is the authority and this page is out of date.

How the module is assembled

The page is `tools/index.html`. Its rail groups the tabs five ways — *On set* (Slate & Takes, Timecards, Offload/MHL, Dailies Review), *The office* (Crew & Call Sheets, Hot Costs, Insurance, Rights / Chain of Title, Finance Waterfall), *Story & look* (Script Revisions, Story Bible, Moodboard, Lens & Coverage, Look / LUTs), *Finishing* (Captions, Credit Roll) and *Selling* (Festivals, Buyers & Investors, Press Kit). A tab builds itself the first time you open it and not before; the current tab is written into the URL fragment, so `#captions` reopens where you left off.

Underneath, the split is deliberate: *lib-* files hold arithmetic that can be tested in node with no browser at all, and *tools-* files hold the screens that call it. `tools/lib-media.js` (TMedia), `lib-sun.js` (TSun), `lib-money.js` (TMoney) and `lib-script.js` (TScript) are the four engines; the four `tools-*-ui.js` files and `tools-registers.js` are the screens.

The header of `tools/tools-core.js` states a hard load order, and it is enforced rather than trusted. The page must load `/js/safe-url.js`, then `/js/ui-chrome.js`, then `/js/ui-table.js`, and only then `tools-core.js`. `TCore` is not a second implementation of anything — it is a lazy re-export of eleven helpers from `CTable` plus `toast` from `CChrome`, each a getter that throws by name if its file is missing, so a broken load order reports the file to add instead of failing three modules later as an undefined function.

The Registers

Eleven of the nineteen tabs are built on one component — thirteen registers in all. A **Register** is defined in `js/ui-table.js` and takes a schema: a storage key, a list of fields, and optionally a summary line, a blank-row shape, an expiry field and a flags function. Each field declares a type — `text`, `money`, `number`, `date`, `select` or `textarea` — and that decides how the cell is edited. Rows persist to `localStorage` under the schema's key on every change; **+ Add** puts a new row at the top. Where a schema names an `expiryField`, that date column grows a chip: days remaining once inside thirty, and **EXPIRED** once the date is past. Insurance keys that to `expiry`, Rights to `termEnd`, Festivals to `deadline`.

REGISTER	STORAGE KEY	WHAT IT HOLDS
Crew directory	`SB_Crew_v1`	Name, role, dept, union, rate, phone, email, dietary, emergency contact
Call-sheet distribution	`SB_CallDist_v1`	Shoot day, recipient, sent via, sent date, status, notes
Insurance & certificates	`SB_Insurance_v1`	Type, carrier, policy no., limits, additional insured, dates, premium
Rights / chain of title	`SB_Rights_v1`	Work, agreement type, counterparty, territory, media, term, fee, status
Buyers & investors	`SB_Deals_v1`	Contact, company, type, territory, stage, value, last contact
Festival submissions	`SB_Festivals_v1`	Festival, category, tier, deadline, fee, result, premiere requirement
Take log	`SB_TakeLog_v1`	Shoot day, time, scene, take, roll, grade, note
Review notes	`SB_ReviewNotes_v1`	Clip, timecode, note, status
Timecards	`SB_Timecards_v1`	Date, name, role, rate, call, wrap, worked, total
Hot-cost postings	`SB_HotCost_v1`	Date, account, description, actual or PO, amount
Story bible	`SB_Bible_v1`	Name, type, one-line, detail, appearances
Investor classes	`SB_FinClasses_v1`	Class, invested, premium, corridor
Deferrals	`SB_FinDefs_v1`	Deferral, amount

> **The Crew register is a file of personal data.** `SB_Crew_v1` holds a full name, telephone number, email address, dietary requirement and emergency contact for every member of your crew. Export CSV writes all of it to `Crew.csv` in plain text, with no password and no encryption. It is also an `SB_*` key, which means the vault sweeps it into every project snapshot and into every `.cinamate` archive you hand to somebody else (`projects/lib-vault.js` excludes only the two keys that carry API credentials). Treat that CSV and that archive as you would a printed crew list: know where the copies are, and do not attach either to anything you would not send to a stranger.

> **Deleting a row asks nothing.** The ✕ at the end of a Register row removes it and rewrites the store immediately — no confirmation, no undo. On the Festivals tab the consequence reaches further: that register is bound to the same store the Festival Strategist reads, so a row deleted at `/tools/#festivals` is gone from `/festivals/` too.

Why the CSV has apostrophes in it

Export CSV writes a header row of the field labels and one row per record, naming the file after the store — `SB_Crew_v1` becomes `Crew.csv`, `SB_Rights_v1` becomes `Rights.csv`. Values are quoted and embedded quotes are doubled, in the ordinary way.

Then there is one further step, and an operator who notices it should know it is deliberate. Excel and Google Sheets both treat a cell that *begins* with `=`, `+`, `-`, `@`, a tab or a carriage return as a formula rather than as text. A location note reading `-2 hours travel`, or a hostile string typed into a field by somebody who knew what they were doing, would therefore execute on the machine of whoever opened the export. `csvSafe` prefixes a single apostrophe to any such cell. Both spreadsheets strip that apostrophe on display, so the text stays readable and stays inert; nothing is removed. The guard is exercised for real — not merely grepped for — by

`scripts/test_csv_injection.mjs`, which drives `Register.toCsv` with seven live formula payloads and asserts both that none survives and that the original text is still legible.

Sun, daylight and weather

`tools/lib-sun.js` implements the standard NOAA/Meeus solar equations, and its own header claims ±2 minutes — ample for a call sheet, not a substitute for an ephemeris. It is the platform's only solar engine: `/locations/`, `/production/`, the Producer Suite and this module all load the same file. `sunTimes(date, lat, lon)` returns six moments for a calendar day: civil dawn and civil dusk (sun at −6°), sunrise and sunset (−0.833°, the refracted horizon), and the two golden-hour boundaries — `goldenEndAM` and `goldenStartPM`, both taken at the sun 6° above the horizon. `solarNoon` gives the day's peak, `daylightHours` the span between rise and set to one decimal, and `sunAngles` the sun's *direction* at each of those moments: an azimuth in degrees clockwise from true north, an altitude above the horizon, and a sixteen-point compass name. That is the number a gaffer actually asks for.

> **Timezone: the tool computes instants, you supply the clock.** Every time above is returned as a UTC instant. `fmtLocal(ms, offsetMinutes)` renders it in whatever offset it is *given*, and if it is given none it silently uses the offset of the machine you are reading on. That is correct only if you are standing on the location. The authoritative offset is the location's own civil offset, read from the Open-Meteo forecast response (`timezone=auto` yields `utc_offset_seconds`) and then remembered on the saved plan. Until a forecast has ever been fetched for that pin, the fallback is an estimate from longitude — `round(lon / 15)` hours — which is solar mean time and knows nothing of political borders or daylight saving. It can be a full hour wrong, in the direction that puts a crew on a hillside at the wrong time. Any screen using the estimate is required to say so, and the day planner labels the column `UTC±hh:mm` with the source spelled out beneath. Read that label before you publish a call time.

There is no separately named "magic hour" in the code. Golden hour is the 6° boundary; the −6° civil dawn and dusk figures are what bracket the shootable twilight either side of it.

Weather is fetched by your browser from the keyless public Open-Meteo API — daily weather code, maximum and minimum temperature, maximum precipitation probability and maximum wind. `wmoLabel` turns the WMO code into English and reports `Code n` honestly for anything it does not know. `shootRisk` scores a day out of 100: precipitation probability weighted 0.6, a maximum wind above 30 in the forecast's own unit adding up to 25 more, plus 30 for a thunderstorm code or 15 for the rain and snow codes between 61 and 86. It is a triage number for reordering exteriors, not a forecast in itself, and a failed fetch is reported as a failure — a blocked request cannot masquerade as "beyond forecast".

Money: timecards, hot costs, waterfall

`tools/lib-money.js` carries three engines. The timecard is the one an operator meets first. Give it an hourly rate, a call and a wrap, and it splits the day under the published twelve-hour conventions held in `TC_DEFAULTS`: 1.5× beyond eight *worked* hours, 2× beyond twelve *elapsed* hours, 3× golden time beyond fifteen elapsed. Note which clock each threshold reads — overtime is measured on worked hours, double and golden time on elapsed, which is the industry convention and the reason meals move one figure and not the other. Up to two meals of thirty minutes each come off the worked total.

The first meal is due within six hours of call; a late or missing meal accrues a penalty per violated half-hour on an escalating scale of 25, 30 then 50, capped at twelve half-hours. Turnaround is ten hours, and a short rest since yesterday's wrap bills the invaded hours at the crew member's rate. A sixth consecutive day multiplies every hour line by 1.5 and a seventh by 2.0. Fringes default to 28% of the subtotal. The engine's docstring is unambiguous that these are published conventions parameterised for your own agreement, and that this is neither payroll nor legal advice.

> **Only one rule is editable on the screen.** `TMoney.timecard` accepts an override for every threshold above, but the Timecards tab wires just one of them: *Fringe %*. The tab's own footnote invites you to "edit the thresholds in the fields above", and the shipped fields do not include them — call, wrap, meals, first-meal hour, day-of-week, yesterday's wrap and fringe are the whole set. A production on a different sideletter should verify the split by hand rather than assume the defaults were changed.

Hot Costs groups postings by top-sheet account code, separates spend already invoiced from purchase orders merely committed, and reports total against budget with *HOT* above 85% used and *OVER* past 100; the budget column seeds from the saved top sheet. **Finance Waterfall** layers instruments on a lifetime revenue figure: deferrals paid first, then each investor class recouping its investment plus its premium, then the remaining pool split by corridor percentages with the balance to the producer and talent pool. A step that cannot be paid in full is shown short rather than rounded away.

Script utilities

`tools/lib-script.js` holds two unrelated things that happen to both be text. The first is a longest-common-subsequence line differ behind **Script Revisions**, which snapshots the Studio's current script as a locked draft and compares any two, generation by generation, in the production colour order: White, Blue, Pink, Yellow, Green, Goldenrod, Buff, Salmon, Cherry, 2nd Blue, 2nd Pink.

The second is captions. `parseCaptions` reads SRT and WebVTT through the same path — it accepts both the comma and the full-stop decimal separator, tolerates a byte-order mark, skips `WEBVTT`, `NOTE` and `STYLE` blocks, and sorts cues by start time. `toSrt` and `toVtt` write either format back out, so the conversion between them is simply parse-then-write. `captionQc` checks each cue against the ordinary broadcast conventions and reports what it finds: reading speed above roughly 20 characters per second, a line longer than about 42 characters, three or more lines in one cue, a cue that overlaps its predecessor, and an end before its start. In the Captions tab, cues live in `SB_Captions_v1`, a line break inside a cue is typed as a space, a vertical bar and a space, and the exports land as `captions.srt` and `captions.vtt`.

Camera media: lenses, LUTs and offload verification

`tools/lib-media.js` is the module's camera maths, and it owns the platform's single sensor table — nine formats, each with its aperture in millimetres, read by the Set Designer's 2D plan and its 3D viewport as well as by the Lens tab, so one lens gives one answer everywhere.

KEY	FORMAT	KEY	FORMAT
`super35`	Super 35, 24.9 × 18.7	`alexa-65`	ARRI 65, 54.1 × 25.6
`super35-17x9`	Super 35 17:9, 24.6 × 13.1	`red-vv`	RED V-RAPTOR VV, 40.96 × 21.6
`fullframe`	Full frame, 36 × 24	`mft`	Micro 4/3, 17.3 × 13
`alexa-lf`	ARRI LF, 36.7 × 25.5	`s16`	Super 16, 12.52 × 7.41
`iphone-main` — phone main camera, ~9.8 × 7.3		Unknown or missing key resolves to `super35`	

From a format and a focal length the Lens tab gives horizontal and vertical field of view, the frame's width and height at your subject distance, and the full-frame equivalent focal length, with a top-down frustum drawn for blocking conversations. Given an aperture, it adds depth of field. The circle of confusion is stated rather than buried: frame diagonal divided by 1500, which reproduces the familiar 0.029 mm on full frame; a house working to a different standard passes its own value instead of editing the file. Distances are metres, and the far limit at or past the hyperfocal is returned as infinity rather than as a large number pretending to be a measurement.

Look / LUTs parses the public IRIDAS/Adobe `.cube` text format and previews it over a still with an intensity blend. It is 3D only — a 1D LUT is refused with a message saying so, and a truncated file is refused naming how many rows were expected against how many arrived.

Offload / MHL is the one to know on a card day. Pick the files off a card and each is SHA-256 hashed in your browser — nothing is uploaded anywhere — into an XML sidecar manifest recording path, size and digest per file. Later, load that manifest and re-pick the copied files: the verifier reports each file as ok, changed, missing or extra, and declares the copy clean only when nothing has changed and nothing is missing. That is the difference between believing a copy and knowing it.

> **What this module does not contain.** There is no media *storage* calculator here. Nothing in `/tools/` holds a codec list, a data rate in megabits, a card capacity or a drive-sizing estimate, and no such table was found anywhere in the shipped tree. Any figure of that kind must come from your camera manufacturer's own tables. The manual will not invent one, and neither should a spreadsheet built from memory the night before a shoot.

The tabs that are neither a register nor a calculator

Slate & Takes is a tappable slate that flashes, sounds a short 1 kHz tone for sync, writes the take into the log and advances the take number. Every logged take carries the shoot day as a field, because a take with no date appears on no daily production report — the summary line counts the undated ones and says so. **Dailies Review** plays a clip locally, steps a frame at a time (at a fixed 1/24 s), lets you draw on the picture and logs notes against the running timecode. **Moodboard** holds dragged and resized reference images with notes, downscaled on import to keep the store manageable, and exports the arrangement as a PNG. **Credit Roll** builds its text from the project title, the script's characters and the Crew directory, previews the scroll at one of three speeds and records it out as a WebM. The **Press Kit** tab assembles title, logline, runtime, synopsis, stills and credits into a single self-contained HTML file you can send on. All of them run entirely in the browser; nothing is uploaded.

What is verified, and how

Four suites cover this module, and they matter because they say which numbers have been checked against something outside the code. `scripts/test_tools.mjs` drives all four libraries — sun times against the NOAA calculator, the diff, the caption round-trip, the timecard split, the hot-cost roll-up, the waterfall, the LUT and the manifest. `scripts/test_media.mjs` checks the sensor table, fields of view against published lens tables, and the depth-of-field algebra — including the definitional test that focusing at the hyperfocal puts the near limit at half of it and the far limit at infinity. `scripts/test_sun.mjs` exists solely because the timezone defect was found three times and survived each time: its fixtures are all remote, and the offset assertions are re-run inside child processes pinned to three different machine timezones, because a suite that only asks about its own clock cannot see that class of bug. `scripts/test_csv_injection.mjs` covers the export guard described above.